SEDust: User Manual

Abstract

This manual describes the model-agnostic dust library under `SEDust/sed/`. It computes the infrared emission spectrum *and* the transport optics (extinction, scattering, asymmetry) of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Mathis, Rumpl & Nordsieck (1977) graphite+silicate model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type, one emission call, and one extinction call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

This version is the **scalar** branch (v1.00): unpolarized cross sections and unpolarized emission. There is no polarized-transfer API here — no dichroic extinction matrix, no aligned-grain scattering matrix. Use v1.20 for those.

Contents

1. Overview

A *dust model* is represented by a single derived type, dust_model_t. You build it once (per RT run), then call dust_emission once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_dust(m, 'astrodust', '../data') ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
   ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
   call dust_emission(m, J_lam, lamI_total) ! per cell
```

```
  ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

One archive:

```
cd SEDust/sed && make libsedust.a # the library archive
```

Nothing in it reaches the T-matrix, so `SEDust/tmatrix` need not be built for the default, non-ionizing Q table (1129 wavelengths, 0.0912 µm to 39810 µm). The separately shipped `q_astrodust_P0.20_Fe0.00_1.400_euv.dat` has 1762 wavelengths and reaches 1×10^{-4} µm; explicit EUV commands select that file. What follows applies to a model that forms a band below the table it was given. Built from this archive, such a model *refuses* the spheroid (`euv_tmatrix = .true.`, the default whenever `lam_min` is given) with `status = 6`, rather than answering with a different particle behind the caller's back; `euv_tmatrix = .false.` asks for the volume-equivalent sphere explicitly.
A host that wants such a band on the spheroid of the Q table (§6.3) builds the T-matrix into the *same* archive, so the link line below does not change:

```
cd SEDust/tmatrix && make libtmatrix.a # T-matrix
cd SEDust/sed && WITH_TMATRIX=1 make libsedust.a
```

and names the spheroid once, before the model is built:

```
use euv_astrodust_tmatrix, only: use_tmatrix_euv_band_optics
call use_tmatrix_euv_band_optics()
```

`SEDust/sed/build_lib.sh` is the same two builds as a standalone script, for an RT tree that carries its own copy of `sed/src/`; it writes `sed/lib/` and takes `WITH_TMATRIX=1` the same way.
Of the CLI drivers, `calc_sed.x astrodust` and `calc_kext.x` are built against `libtmatrix.a` and make that call themselves — they are the two that would compute an astrodust EUV band if one ever formed. `calc_sed.x dl07` and `calc_sed.x zubko` need no T-matrix: those models' optics reach past the Lyman limit on their own.
The compiler is `gfortran`. The default flags are `-O3 -ffree-line-length-none -fopenmp` plus a warning set — deliberately *portable*, so that a `libsedust.a` built on one node runs on any other. A `-march=native` line is present but

commented out; enabling it can change the last digits through FMA contraction. `libsedust.a` is a static archive of all library objects; the Fortran `.mod` files are written alongside it in `SEDust/sed/`. The `Makefile` declares `.NOTPARALLEL`, because every executable recipe compiles the shared module sources into the same directory and `make -j` would race on the `.mod` files.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model's output channels (Table 1).

builder	model	channels	notes
`build_astrodust`	HD23 astrodust+PAH	`AD`, `PAH`	N_C=417, D16 graphite
`build_dl07`	Draine & Li 2007	`SIL`, `CARB`	N_C=470, D03 graphite
`build_mrn`	Mathis, Rumpl & Nordsieck 1977	`GRA`, `SIL`	$a^{-3.5}$, no PAH
`build_zubko`	Zubko (ZDA 2004) BARE-GR-S	`PAH`, `GRA`, `SIL`	neutral PAH only
`build_dustem`	THEMIS (Jones+13, Köhler+14)	one per `GRAIN` line	DustEM input files
`build_dustem`	Guillet et al. (2018) Model D	one per `GRAIN` line	no $\langle\cos\theta\rangle$
`build_from_files`	file-defined	`CH1`, `CH2`, ...	generic loader

Table 1: Built-in model builders. Each sets the model's own optics/abundance parameters internally. `build_dustem` serves both DustEM-defined models; which one it builds is the `GRAIN_*.DAT` it is given, and the channels are that file's grain populations in its own order — the layout of DustEM's `EXT_`/`SED_` products.

Signatures (all optional arguments in brackets; every optional is a keyword argument in the order shown). `build_dust` is the one entry point — it takes the model by name and a data directory and resolves the rest from the HDF5 products (§12.); the builders below it stay exported so that a host naming its own files keeps working:

```
call build_dust (m, model, data_dir [, NT, T_lo, T_hi] [, include_euv] &
                  [, status] [, message] [, lam_min] [, kext_path] [, sd_index] [, u_isrf] &
                  [, config_path] [, astrodust_index_path] &
                  [, euv_tmatrix])
call build_astrodust(m, qtable_path, NT, T_lo, T_hi &
                  [, status] [, lam_min] [, astrodust_index_path] [, kext_path] &
                  [, euv_tmatrix] [, include_euv])
call build_dl07 (m, qtable_path, sd_index, U, NT, T_lo, T_hi &
                  [, status] [, lam_min] [, kext_path] [, lam_axis] &
                  [, include_euv] [, stored_q_dir] [, pah_xsec])
call build_mrn (m, qtable_path, NT, T_lo, T_hi [, status] [, lam_min] &
                  [, kext_path] [, lam_axis] [, include_euv] &
                  [, stored_q_dir])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status] [, kext_path] &
                  [, lam_min] [, include_euv] [, qtable_path] [, optics])
```

```
call build_dustem (m, grain_path, data_dir, NT, T_lo, T_hi [, status] [, kext_path] &
                   [, lam_min] [, include_euv] [, gsca_missing] [, qtable_path])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status] &
                   [, kext_path] [, lam_min] [, include_euv])
```

- `NT`, `T_lo`, `T_hi`: number and range [K] of the internal temperature grid (typical `200, 2.7, 5000`).
- `sd_index` (DL07): WD01 size-distribution index — `7` = MW R_V=3.1, $q_{\rm PAH}$=4.58%; `29/30/31` = LMC2; `32` = SMC.
- `U` (DL07): ISRF intensity scaling (the DL07 reference uses U=1).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover. `build_dust` maps every builder's own stage number onto *one* vocabulary, so a host on the single entry point does not branch on the model name to read a code, and takes an optional `message` carrying the reason in words — what an MPI host prints before a collective abort.
- `data_dir` (`build_dust`): the data root for the whole library while the build runs. The dielectric functions, the default extinction curves and the stored cross-section tables all resolve inside it, so a host may call `build_dust` from any working directory with an absolute path, and a file that cannot be read arrives as a `status`, never as a runtime abort. A path the caller names itself is used as given and never composed with the root. A host that calls the builders directly sets the root once with `sed_set_data_root`, re-exported from `dust_lib`.
- `stored_q_dir` (optional, DL07 and MRN): where this model's stored cross sections come from. Omitted, the model's own directory under the data root, with the `/qtable` of an HDF5 `qtable_path` ahead of it. Passed blank, nothing stored is read and every optic is solved from the dielectric functions.
- `pah_xsec` (optional, DL07): which vintage of the PAH cross sections the carbonaceous channel is built on, `'dl07'` (default, the DL07 paper's own prescription) or `'ld01'` (Li & Draine 2001); it is what the `ld01` keyword of the CLI drivers sets. Any other string is refused with `status = 8` rather than defaulted.
- `qtable_path` (optional, Zubko): the model's HDF5 product, which is where its stored cross sections then come from. This is an argument rather than hidden state so that a program writing `/kext` into a product takes its optics from the `/qtable` of that same product.
- `lam_min` (optional, astrodust, DL07 and MRN): shortest wavelength [μm] the model must cover, for a host that transports ionizing radiation. See §6.. **Omitting it reproduces the previous behavior exactly** — the grid, and every number derived from it, is unchanged. For astrodust what it does depends on the Q table: on the default 1129-wavelength one a floor below 0.0912 μm builds a band, while the `_euv` table already reaches the short end of the DH21 dielectric function, so there every value that is not refused falls inside the table and prepends

nothing. DL07 can be carried to 6.205×10^{-5} μm below either, because its D03 optical constants reach past the table.

- `astrodust_index_path` (optional, astrodust only): the dielectric function the model is made of. It is read to decide how far `lam_min` may reach, and it would supply the optics of a band below the Q table if one ever formed. It must be the file `qtable_path` was computed from — same porosity, iron fraction and axial ratio — or the model changes material at the seam. Omitted, it is the default that pairs with the shipped Q table:

```
data/dielectric/index_DH21Ad_P0.20_0.00_1.400 <->
data/astrodust/q_astrodust_P0.20_Fe0.00_1.400.dat
```

- `kext_path` (optional, all builders): the precomputed size-integrated extinction table this model will serve through `dust_extinction` (§5.). The builder loads it, because the path is relative to `sed/` and a host is free to change directory once the model is built. Naming a file is a contract: if it cannot be read the *build fails*. Omitted, the model's default table (Table 2) is tried, and a default that is not there leaves the model with no extinction to serve — but the build succeeds, so a driver that only computes emission still runs.

- `euv_tmatrix` (optional, astrodust only, default `.true.`): how the extended band's optics are computed. `.true.` is the T-matrix on the same $b/a = 1.400$ oblate spheroid as the Q table, and requires a library built with the T-matrix and the one registration call of §2. — without them the build fails with `status = 6` instead of substituting a sphere silently. `.false.` asks for the volume-equivalent-sphere Mie approximation, which is far cheaper and about 2% low in the geometric-optics limit, and needs no T-matrix at all. See §6.3. Ignored when `lam_min` asks for no extension — which, with the shipped Q table, is every legal request, so **this argument currently selects nothing**. It is kept for a model whose optics table stops short of its own dielectric function.

3.1 The MRN model

`build_mrn` is the classical Mathis, Rumpl & Nordsieck (1977) mixture: graphite and astrosilicate spheres, one power law per material,

$$\frac{dn_i}{da} = A_i \, n_{\rm H} \, a^{-3.5}, \qquad 0.005\ \mu{\rm m} \le a \le 0.25\ \mu{\rm m}, \tag{1}$$

cut sharply at both ends. Equation (1) is Draine & Lee (1984, eq. 5.1); the cutoffs are MRN's own estimates, which DL84 held fixed. There are *no* PAHs in it — the smallest grain is a 50 Å graphite sphere and the 2175 Å feature is graphite's — so the emergent SED carries no aromatic emission features at all. That is the model, not a gap in the solve.

The normalization. $\log_{10} A_{\rm gra} = -25.16$ and $\log_{10} A_{\rm sil} = -25.11$, in cm$^{2.5}$/H: the abundances DL84 §Va adopted after fitting the Savage & Mathis average extinction curve with their own dielectric functions. They are what Draine's own published curve `kext_albedo_MRN` is the size integral of — his parameter file for it states the grain volumes per H, 2.49×10^{-27} and 2.79×10^{-27} cm^3/H, and integrating $(4\pi/3)a^3 A_i a^{-3.5}$ over $[0.005, 0.25]$ μm turns those into exactly

builder	default `kext_path`
`build_astrodust`	`../data/astrodust/kext_astrodust_MW_euv.dat`
`build_dl07`	`../data/dl07/kext_dl07_MW_euv.dat`
`build_mrn`	`../data/mrn/kext_mrn_euv.dat`
`build_zubko`	`../data/zubko/kext_zubko_BARE_GR_S_euv.dat`
`build_dustem`	`<data_dir>kext_<model>.dat`
`build_from_files`	none

Table 2: Default extinction table of each builder. The EUV product is the default wherever the model has one, because its wavelength range *contains* the plain grid's: the two are the same size integral on the same optics, the extended one merely carried further into the ionizing band, and their nodes coincide over the overlap. One default therefore serves a host that transports ionizing radiation and one that does not; it is the *narrower* non-EUV table that has to be named explicitly. The astrodust pair differs by the 633 wavelengths the `_euv` Q table carries below the Lyman limit; the DL07 pair by those and by the further 61 points its D03 optical constants reach past the table, and the MRN pair by the same 694, its optics being that same calculation; the Zubko pair by 335 — its own ZDA tables carry 336 nodes below that limit and the narrow product keeps the last of them, 0.089 98 μm, so that it still covers the band a host transports — and there the wide product is the tables' own range and the narrow one is that range cut, the reverse of the other two: the Zubko model needs no EUV *extension* and could not have one, its own optics table already reaching 1.24 keV.

these two A_i. That curve ships as `data/release/kext_albedo_MRN` and `calc_kext.x mrn` prints the point-by-point comparison against it at the end of every run.

The same sentence of DL84 quotes MRN's *own* pair, $A_{\rm sil} = 10^{-25.10}$ and $A_C = 10^{-25.13}$ after adjustment to the Bohlin, Savage & Drake (1978) $N_{\rm H}/E(B-V)$. They differ by 7% in the graphite abundance and are *not* offered here: this model is the one the reference curve tests.

Optics, enthalpy and grid. Mie on the Draine (2003) dielectric functions — the same `q_silicate_full` and `q_graphite_full` the DL07 model's silicate and graphite come from, graphite on the random-orientation average $\frac{1}{3}E\parallel c+\frac{2}{3}E\perp c$. MRN themselves used Wickramasinghe's optical constants; DL84 recomputed this size distribution on theirs, and D03 is the current revision of those. The enthalpies are the Draine & Li (2001) heat capacities, and the densities 3.5 and 2.2 g cm^{-3} — the values those capacities and the D03 optics are both defined with. Because the optics are dielectric-function Mie at every wavelength, the model takes only a wavelength *axis* from its product, exactly as DL07 does, and `lam_min` extends the grid rather than the physics. The radius grid is the model's own: 70 log-spaced points with the two cutoffs of eq. (1) on its ends.

What the comparison against Draine's curve shows. From the far ultraviolet through ~ 20 μm — the whole band the size distribution was fitted in — our $C_{\rm ext}$/H agrees with `kext_albedo_MRN` to 1–2%. Longward of ~ 60 μm ours is lower, by 22% at 100 μm and 18% at 1000 μm. That is an optical-constants vintage difference and not a defect here: Draine's MRN table is dated December 2002 and its silicate is his `ICOMP=7` "smoothed UV silicate", while ours is the

2003 astrosilicate. His own two curves for the *same* power law and the same $a_{\max}$ at essentially the same total volume per H — the 2002 MRN table against the D03 `amax_0.25um` one — differ by +23% at 100 μm and +15% at 1000 μm in the same direction. Our D03 optics are the ones checked against his D03-vintage DL07 table, which they reproduce to 0.01% in the same band (§5.).

Energy is conserved. Solved with `equil` the model emits 0.99987 of what it absorbs from the Mathis field, and with `draine` 1.00003 — the same 10^{-4} closure the DL07 model gives (0.99991). Under the default `heuristic` solver it emits 0.949, against DL07's 0.977; that shortfall is the heuristic solver's own accuracy on this size distribution, whose smallest grains are stochastically heated 50 Å graphite spheres, and not a property of the model (§4.2).

Two routes to the Zubko model. `build_zubko` evaluates the ZDA log-polynomial size-distribution *formula* (coefficients read from `ZDA_BARE_GR_S_Config.dat`) on the optics-table radii; `build_from_files` with `data/zubko/zubko_descriptor.txt` instead interpolates the *tabulated* `SzDist` files onto the same radii. The two shapes agree to $\sim 10^{-5}$, but the distributed tables carry a normalization that sits below $A\,g(a)$ of the config by 0.79% (PAH), 1.13% (graphite) and 0.41% (silicate), and they are sampled at only 24 / 62 / 63 radii, so interpolation near $a_{\max}$ adds a little more. Together this moves the size-integrated $C_{\rm ext}$ by up to 2.10% (median 1.02%). **The formula is the default and is faithful to the ZDA definition**; the tabulated route exists to reproduce the benchmark's own size grid.

The ZDA optics are a homogeneous-sphere Mie calculation, and that was checked. The three ZDA optics tables are the model definition, so nothing is recomputed at run time. What they *are*, however, is stated in two independent places and was verified here. Zubko et al. (2004, §3) write that for the bare silicate, graphite and amorphous-carbon particles "the extinction and absorption cross sections were calculated using Mie theory for spherical dust particles (Bohren & Huffman 1983)" — the effective-medium step of that paper applies to its *composite* models only. Each shipped file repeats it: the silicate and graphite headers name the dielectric function they were computed from (Draine's `eps_Sil` and `eps_Gra`), while `PAH_28_1201_neu.dat` names the Li & Draine (2001) and Draine & Li (2007) absorption cross sections instead — that component is not Mie and has no refractive index to be recomputed from.
Recomputing `suvSil_121_1201.dat` with the library's own Bohren–Huffman Mie on Draine's published `eps_Sil`, over all 121×1201 cells:

- at a fixed wavelength the ratio $Q^{\rm ours}/Q^{\rm table}$ is *independent of grain radius to four digits* across the whole 3.55×10^{-4}–100 μm range (e.g. 1.0171 at 9.9 μm and 1.0067 at 99.8 μm at every radius);
- $\langle\cos\theta\rangle$ agrees to $\sim 10^{-4}$.

Both are properties of the sphere and the size parameter, so they settle the question the comparison was asked: the table is a homogeneous-sphere Mie calculation and the library reproduces it. What is left is a purely wavelength-dependent factor — mean $|\Delta Q_{\rm abs}| = 0.59\%$ over 10–100 μm, 1.7% over 1–10 μm, rising at the extreme long-wavelength end — and a radius-independent, wavelength-dependent residual can only come from the refractive index, not from the scattering solution. The published `eps_Sil` is tabulated on a different wavelength grid from the table (200 per decade, ending at 10^3 μm, against 171 per decade to 10^4 μm), so the file the table's generator actually evaluated was a resampled

and extended copy that is not the one distributed. Reproducing the tables to better than a percent would need that copy; nothing in SEDust depends on it, because the tables themselves are what every Zubko build reads.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_\lambda / N_\mathrm{H}$ [erg s^{-1} cm^{-2} sr^{-1} H^{-1}] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (`m%stoch_method`)

The field `m%stoch_method` selects how each grain is heated (Table 3); `dust_emission` honors it.

`m%stoch_method`	method	note
`'heuristic'`	GD look-ahead T-window narrowing	**default**; fastest, serial
`'draine'`	Guhathakurta & Draine (1989)	reference GD
`'qm'`	energy-space transition matrix (thermal-discrete)	most detailed
`'equil'`	single-grain equilibrium (no stochastic)	see §4.3

Table 3: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); `'equil'` forces equilibrium.

The cooling kernel of `'qm'` (`qm_method`). `'qm'` names the energy-space transition matrix; *which* downward-transition rate fills it is a second and independent selector, the module variable `qm_method` of `stoch_qm_mod` (`use stoch_qm_mod, only: qm_method`), set once before the solve. It is module state, not a field of `m`, and the other three solvers never read it.

qm_method	downward transition rate
'dbdis'	thermal-discrete: every transition $f < i$ retained. **Default**, and the production reference.
'dbcon'	thermal-continuous: the downward rates are collapsed onto the nearest-neighbor transition, which reproduces the continuous-cooling approximation of the Guhathakurta & Draine temperature-space recursion.
'stati'	exact-statistical (microcanonical): the rate carries the degeneracy ratio g_f/g_i in place of a Planck factor, with $g(U)$ counted exactly by the Beyer–Swinehart algorithm over the vibrational mode spectrum, and the emission is the degeneracy-weighted kernel rather than a bin blackbody. No bin temperature enters it at all.

'stati' needs the full mode spectrum and a state count that stays representable, so it is applied only to PAH grains with $a \leq 25$ Å and reverts to 'dbdis' for every other grain — silicate included, as in Draine's own production, whose silicate 'stati' branch is flagged untested. For a silicate grain the counted $\ln g$ that survives the feasibility guards still carries a residual non-monotonicity of order 14 in the log, which the $\exp(\ln g_k - \ln g_j)$ ratio amplifies into a spurious mid-infrared spike. Where it does run, 'stati' agrees with 'dbdis' to within a few percent in every band. An unrecognized qm_method stops the run rather than being defaulted.

Sizing the transition matrix. Two further module variables of stoch_qm_mod set its dimensions, and again only 'qm' reads them: qm_nstate_default, the number of enthalpy bins (default 200), and qm_nisrf_max, the cap on how many radiation-field wavelengths the matrix is built on (default 200). Draine's production settings are 500 bins and the full 2500-point field, with no downsampling. The CLI drivers expose the kernel and both sizes as qm_dbcon / qm_stati, nstate=N and nisrf=N (§7.2).

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set m%stoch_method = 'equil' and call dust_emission. Every grain is placed at its own equilibrium temperature $T_{\rm eq}$, computed from *that grain's* absorption cross section, and emits $C_{\rm abs} B_\lambda(T_{\rm eq})$. No stochastic excursions are computed.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C^{\rm tot}_{\rm abs}(\lambda) = \sum_{\rm pop} \sum_a {\rm d}n(a) C_{\rm abs}(\lambda, a)$. The single $T_{\rm eq}$ satisfies $\int C^{\rm tot}_{\rm abs} J \, {\rm d}\lambda = \int C^{\rm tot}_{\rm abs} B(T_{\rm eq}) \, {\rm d}\lambda$, and the emission is $C^{\rm tot}_{\rm abs} B_\lambda(T_{\rm eq})$. The optional Teq_out returns that temperature.

Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U, precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

`J_ref(m%NLAM)` is the reference field shape (at U=1) and `U_grid(:)` an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U. The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final `status` (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times$ (fixed $J_{\rm ref}$). For cells whose field *shape* departs from $J_{\rm ref}$ (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final `status` argument (`integer, intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints a message and `stop`s the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply `0` = built, nonzero = build failed.
`build_dust` carries a vocabulary of its own, and maps each builder's stage number onto it, so a host on the single entry point never branches on the model name to read a code. It also takes an optional `message` (`character, intent(out)`) carrying the reason in words, which is what an MPI host prints before a collective abort:

`0`	built	`6`	calorimetry
`1`	optics table	`7`	grid inconsistent between components
`2`	size distribution	`8`	EUV spheroid optics unavailable
`3`	dielectric function	`9`	model definition (config / descriptor)
`4`	`lam_min` not coverable	`90`	model name not one it codes
`5`	extinction table	`91`	`from_files` without `config_path`
		`92`	`zubko_optics` neither `'zda'` nor `'mie_d03'`

The builders keep their own numbering, listed below, for a host that calls them directly.

- `dust_emission`: `1` unknown `stoch_method`; `2` `'qm'` selected but a population's radii are unset; `3` `m` is not the most recently built model. This routine solves through `sed_grain_loop`, which reads the module grids the last build filled, so it can answer for that model and no other; a stale model used to be answered with another model's

numbers in silence. `dust_extinction` and `size_integrated_extinction` are *not* restricted — both read only the model argument.

- `dust_extinction`: 1 an output array is not of size `m%NLAM`; 2 no extinction table was loaded for this model; 3 `m%lam` reaches outside the loaded table's wavelength range.
- `size_integrated_extinction`: 1 an output array is not of size `m%NLAM`. It reads no table, so it has no counterparts of `dust_extinction`'s 2 and 3.
- `dust_build_table`: 1 `size(J_ref)`$\neq$`m%NLAM`; 2 `size(U_grid)` < 2; 3 `U_grid` not positive and strictly increasing.
- `dust_emission_interp`: 1 `U`≤ 0; 2 `size(lamI_total)`$\neq$`tab%NLAM`; 3 `lamI_chan` present but not `(tab%NLAM, tab%n_channel)`.
- `build_astrodust`: 1 Q-table load failed; (2 is retired: the HD23 size distribution is analytic and nothing is read); 3 the astrodust dielectric function failed to load (raised only when `lam_min` asks for the EUV band); 4 `lam_min` is shorter than that dielectric function's own shortest wavelength (1.00003×10^{-4} μm), which is refused rather than served with a refractive index frozen at the table boundary; 5 an explicitly named `kext_path` failed to load; 6 `euv_tmatrix = .true.` but the spheroid optics of that band are not available — either the library was built without the T-matrix and no `euv_band_optics_i` is registered (§2.), or the registered one reported that it could not compute the band.
- `build_dl07`: 1 Q-table load failed; 4 `lam_min` is shorter than the D03 dielectric functions (6.1992×10^{-5} μm — the longer-reaching of astrosilicate and graphite), refused for the same reason (there is no code 3 on this path: this model needs no separate EUV dielectric function, because its optics are D03 Mie at every wavelength); 5 an explicitly named `kext_path` failed to load; 8 `pah_xsec` is neither `'dl07'` nor `'ld01'`. Code 8 is reachable only by calling `build_dl07` directly: `build_dust` does not forward that argument, so no `build_dust` code corresponds to it.
- `build_mrn`: 1 the wavelength axis could not be read; 2 `lam_min` is shorter than the D03 dielectric functions (6.1992×10^{-5} μm), refused for the same reason as in `build_dl07`; 3 an explicitly named `kext_path` failed to load. This builder reads no size-distribution file — its distribution is eq. (1) — and has no polarized optics, so it needs only these three.
- `build_zubko`: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed; 8 `optics` is neither `'zda'` nor `'mie_d03'`; 6 an explicitly named `kext_path` failed to load; 7 `lam_min` is shorter than this model's own optics table, which it cannot extend.
- `build_dustem`: 1 `GRAIN_*.DAT` unreadable, or it names a size-distribution keyword this code does not implement; 2 `oprop/LAMBDA.DAT` read failed; 3 a population's `oprop/Q_<gtype>.DAT` read failed; 4 a population's size distribution could not be formed; 5 a population's model radii fall outside its optics tables; 6 a population's `hcap/C_<gtype>.DAT` read failed, or does not cover that population's model radii; 7 a population's

oprop/G_<gtype>.DAT exists but could not be read; 8 an explicitly named kext_path failed to load; 9 lam_min is shorter than the DustEM wavelength grid, which this model cannot extend.

- build_from_files: 1 descriptor open; 2 too many pop: lines; 3 invalid channel; 4 no pop: lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed; 9 an explicitly named kext_path failed to load; 10 lam_min is shorter than the model's own optics tables.

A *default* extinction table that cannot be read is not an error on any of these paths: the model is built without one, and the failure surfaces later as dust_extinction status 2.

5. Computing extinction

dust_extinction is the extinction counterpart of dust_emission: a transfer code takes its opacity from the same model object, and on the same wavelength grid m%lam, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, albedo] [, status])
```

- Cext, Cabs, Csca (m%NLAM): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$.
- gbar(m%NLAM) (optional): the scattering-weighted asymmetry $\langle\cos\theta\rangle$ as the table carries it, which is 0 where nothing scatters and may be slightly negative in the far infrared.
- albedo(m%NLAM) (optional): $C_{\rm sca}/C_{\rm ext}$, and 0 where $C_{\rm ext}$ underflows to zero. It is derived here rather than by the caller so that every host gets the same convention at those wavelengths.
- status (optional): error report; see §4.5. It is 0 on success, 1 if an output array is not of size m%NLAM, 2 if no extinction table was loaded for this model, and 3 if m%lam reaches outside the table.

5.1 What the call returns, and where it comes from

dust_extinction *reads* the size-integrated curve from the precomputed table its builder loaded (m%kext_path; see the kext_path argument in §3. and Table 2). Nothing is integrated over grain size at call time. The size integral itself is the separate routine size_integrated_extinction, which has the same argument list, is what calc_kext.x calls, and is what wrote those tables:

$$C_{\rm abs}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm abs}(\lambda,a),$$

$$C_{\rm sca}(\lambda) = \sum_{\rm pop}\sum_{a} \mathrm{d}n(a)\,C_{\rm sca}(\lambda,a),$$

$$\langle\cos\theta\rangle = \frac{\sum \mathrm{d}n\,C_{\rm sca}\,g}{\sum \mathrm{d}n\,C_{\rm sca}},$$

a plain binned sum over every population, since $\mathrm{d}n(a)$ already carries the bin width. It is exported too, for a host that would rather redo the integral than read the product; dust_extinction is the route that gives an RT host exactly the numbers that were checked against the HD23 release and Draine's own kext_albedo tables (§6.7).

Why the builder holds the table. The table paths are relative to `sed/`, and a host typically changes directory back as soon as the model is built — so the file has to be opened while the builder still runs, not when `dust_extinction` is called. That is the whole reason `kext_path` is a builder argument.

Interpolation onto `m%lam`. Needed only where the table grid and the model grid differ. $C_{\rm ext}$, $C_{\rm abs}$ and $C_{\rm sca}$ are positive and locally power-law in λ, so they are interpolated linearly in $(\ln\lambda, \ln C)$; $\langle\cos\theta\rangle$ changes sign and has no logarithm, so it is linear in $\ln\lambda$. Where a model wavelength *coincides* with a table wavelength the table value is returned unchanged, which is the normal case for the shipped defaults — the EUV table's nodes are the plain grid's nodes over the overlap. Outside the tabulated range there is no optics to serve, so the call is refused (`status = 3`) rather than extrapolated.

Two quantities are not read straight from the file. `albedo` is re-derived as $C_{\rm sca}/C_{\rm ext}$ from the interpolated cross sections instead of from the file's own albedo column, because the cross sections are written with more digits (the oldest product rounds its albedo column to six decimals, which floors to 0 in the far infrared). `gbar` *is* taken from the table as it stands: it is already the scattering-weighted average, and re-weighting it would need the size-resolved optics the table does not carry.

Every model scatters. Every builder populates the extinction-side scattering optics, so the `albedo` and `gbar` in every table are physical:

- **astrodust**: $C_{\rm sca}$ and $\langle\cos\theta\rangle$ from the random-orientation T-matrix Q table, at every wavelength of the model.
- **DL07**: Mie on the D03 astrosilicate and graphite dielectric functions (`q_silicate_full` / `q_graphite_full`), on the same random-orientation average ($\frac{1}{3}E\parallel c+\frac{2}{3}E\perp c$) as the absorption. The two carbonaceous charge states share one scattering description: charge shifts PAH absorption features, not the graphite sphere's scattering.
- **MRN**: the same two calculations, on this model's own 70-point radius grid.
- **Zubko / file-defined**: the $Q_{\rm sca}$ and g columns of the model's own ZDA optics tables, read from the same file as $Q_{\rm abs}$. Those tables are a *homogeneous-sphere* Mie calculation — Zubko et al. (2004, §3) state it for the bare grains, and each file's header names the dielectric function it came from, Draine's `eps_Sil` and `eps_Gra`; the effective-medium step of that paper applies to its composite models, not to BARE-GR-S. The PAH file is not Mie at all: its header names the Li & Draine (2001) / Draine & Li (2007) absorption cross sections. §3.1 records what recomputing the silicate table with our own Mie gives.

A population that genuinely does not scatter leaves its scattering optics unallocated and contributes zero. No population of any shipped model is that case. The PAHs come closest, since a PAH has a published absorption cross section and no scattering one, but what scatters in a PAH population is its graphite fraction $\xi_{\rm gra}(a)$ of HD23 equation (15), and the model carries it: dropping it costs 3.2% of $\tau_{\rm sca}$ at $0.1\,\mu$m against the HD23 release and nothing longward of $0.3\,\mu$m.

5.2 Dust mass per H, and mass opacity

Everything above is *per H nucleon*. A host that carries a dust mass density instead needs the model's own dust mass per H to convert:

```
Mdust_H = dust_mass_per_H(m) ! [g/H], one wavelength-independent constant
Kabs = Cabs / Mdust_H ! [cm^2/g], Draine's mass opacity
```

It is the model's own size distribution weighted by the solid density of each population's material,

$$\frac{M_{\rm dust}}{N_{\rm H}} = \sum_{\rm pop} \rho_{\rm bulk} \sum_{a} \tfrac{4}{3}\pi a^3 \, {\rm d}n(a),$$

with a the effective radius in cm and d$n(a)$ the same binned number per H the size integral above uses, so no da enters the sum. The density lives on the population (`m%pops(ip)%rho_bulk`, §8.); a population whose model states none leaves it at 0 and contributes no mass. The function reads `m` only — it needs no radiation field and no built optics beyond the model itself.

Every builder states its densities, so the quantity is defined for every model in the tree:

model	grain densities [g/cm^3]	$M_{\rm dust}/N_{\rm H}$ [g/H]
astrodust	astrodust 2.74, PAH 2.0 (HD23)	1.185×10^{-26}
DL07	astrosilicate 3.5, graphite 2.2	1.750×10^{-26}
MRN	astrosilicate 3.5, graphite 2.2	1.525×10^{-26}
Zubko	each component's own ZDA optics header (3.5, 2.24, 2.24)	1.444×10^{-26}
file-defined	the `pop:` line's `rho`, else the optics file's	—

The astrodust numbers are Hensley & Draine (2023): $\rho_{\rm Ad} = 2.74$ is the porosity-corrected solid density of the astrodust material, and $\rho_{\rm PAH} = 2.0$ is the density behind that model's $N_{\rm C} = 417\,(a/{\rm nm})^3$. The DL07 numbers are stated by that model's own paper, Draine & Li (2007) §2, in the paragraph defining the effective radius $a = (3M/4\pi\rho)^{1/3}$: amorphous silicate 3.5 (also the density the DL01 astrosilicate enthalpy is built on), and carbonaceous grains "a mass density due to graphitic carbon alone of $\rho = 2.2$ g cm^{-3}". That is deliberately *not* the 2.24 of WD01 and LD01, which is the density implied by the carbon-atom count $N_{\rm C} = 470\,(a/{\rm nm})^3$ the DL07 builder selects; DL07 kept that count and quoted the rounder density, and the model being built here is DL07's. The DL07 carbonaceous grains are one material sequence, PAH-like at small a and graphite-like at large a with no size at which the solid changes, so the one density is used across the whole sequence. That the astrodust PAHs take 2.0 and the DL07 carbonaceous grains 2.2 is a difference between the two *models*, not a correction of one by the other. The Zubko densities are read out of the `Density` entry in the header of each component's own ZDA optics table.

Check against DL07, and a caution about his data files. Table 3 of Draine & Li (2007) lists $M_{\rm dust}/M_{\rm H}$ for each of its physical dust models. Model $j_M = 7$, `MW3.1_60`, is the size distribution this builder selects, and the paper gives 0.0104. Ours is 1.750×10^{-26} g/H, i.e. 0.01046 — agreement to 0.55%.

Draine's `kext_albedo_WD_MW_3.1_60_D03.all` header, for the same model, instead states $M_{\rm dust}/N_{\rm H} = 1.870 \times 10^{-26}$ g/H, which is $M_{\rm dust}/M_{\rm H} = 0.0112$ and therefore 7.4% away from his own Table 3. The same file disagrees with the paper on the silicate normalization as well: the header says the WD01 abundances were reduced by 0.93, while the caption of the paper's Fig. 11 says 0.92.

The position taken here is to follow the *file* for the optics and the *paper* for the mass. The optics are not in question — our $C_{\rm abs}/{\rm H}$ agrees with the 2003 table to better than 0.5% from 100 to 10^4 μm, where absorption is in the Rayleigh

limit and therefore proportional to the grain volume, so the material volumes agree with his. What is left is the $\sim 1\%$ that the two normalizations differ by, and it is Draine's own internal inconsistency, not ours. It is worth knowing that $K_{\rm abs}$ from any of these tables inherits whichever convention was used: $C_{\rm abs}/{\rm H}$ is the better-determined quantity, and $K_{\rm abs}$ is that number divided by a convention.

6. The extreme-ultraviolet extension

The astrodust model's wavelength grid is the grid of the T-matrix Q table it was built from, and *two* tables ship side by side:

file	wavelengths	range [μm]
`q_astrodust_P0.20_Fe0.00_1.400.dat`	1129	0.0912 – 39810
`q_astrodust_P0.20_Fe0.00_1.400_euv.dat`	1762	10^{-4} – 39810

The second spans 12.4 keV down to 0.031 eV; the first is that file with every wavelength shortward of the Lyman limit dropped (`make lyman_cut` in `tmatrix/`), row selection only, so over the 1129 they share they are the same numbers cell for cell. **Which one the host passes as `qtab` decides what its model covers.** Given the `_euv` table, the ionizing band is inside the table, computed the same way as the rest of it, so a host that transports ionizing radiation reads it off the table and needs nothing more — in particular it does not have to link `libtmatrix.a`. Given the default table, a model that needs the band builds one, and §6.3 is where its optics come from.

Below $0.0912\,\mu$m the `_euv` table's wavelengths are the DH21 dielectric function's own energy nodes rather than a resampling of them. That band is full of absorption edges, each tabulated as a step across a close pair of energies — the separations run from 2.4×10^{-6} to 3.6×10^{-4} in relative energy, and k jumps by +211% at Fe K (7124 eV), +131% at O K (538), +67% at Fe L (724), +30% at C K (291), +27% at Si K (1846) and +26% at Mg K (1311). Counting them takes a threshold, and the answer moves with it: 23 places where k jumps by more than 0.5% across a pair closer than 5×10^{-4}, of which 14 are closer than 10^{-4}. A uniform 200-per-decade axis would average every one of them into a ramp. Reusing the nodes keeps every edge an exact step between adjacent grid points, and makes the refractive index at each point the tabulated one rather than an interpolate.

One point is deliberately not a dielectric node: $0.0912\,(1-10^{-4})$, just below the Lyman limit. It is placed for the *radiation field*, not for the grain — see §6.1.

Both `build_astrodust` and `build_dl07` take an optional `lam_min` [μm] for a host that needs to reach below the table it was given:

```
call build_dl07(m, qtab, NT, T_lo, T_hi, lam_min=6.21e-5_wp)
```

Log-spaced points are then prepended from `lam_min` up to the table, at a step no coarser than the table's own $\Delta\ln\lambda$ at its short-wavelength end (0.00794 on the `_euv` axis, 0.01156 on the other), with `m%lam(1)` set to `lam_min` exactly. The request is refused (`status = 4`) when it asks for a wavelength the model's own dielectric data does not cover, rather than served with a refractive index frozen at the table boundary. **Omitting `lam_min`, or passing one the table already covers, prepends nothing.**

What `lam_min` does for astrodust therefore depends on which table the model was built from. On the default 1129-wavelength table a `lam_min` below 0.0912 μm builds a band. On the _euv table there is nothing left to ask for: it already reaches the short-wavelength end of the DH21 dielectric function, so every `lam_min` either falls inside the table (no band) or below the dielectric data (refused). DL07 and MRN can be extended below either, to 6.205×10^{-5} μm, because the D03 optical constants their optics come from reach further than both.

6.1 A step in the field needs a grid point, exactly as a step in the material does

An interstellar radiation field is exactly zero below the Lyman limit and finite above it: `J_Mathis` in `sed/src/radfield.f90` opens with `if (lambda(i) < 0.0912d0) J(i) = 0`, and immediately above the limit $J_\lambda = 3069\,\lambda^{3.4172}$ is 0.856. That is a step discontinuity of the *field*.

The DH21 dielectric energy axis has no node between 13.595 and 14.000 eV. Built from those nodes alone, the grid would jump from 0.088557 μm straight to 0.0912 μm — one cell 2.98% wide, sitting across the step. Every wavelength integral in the library is a trapezoid sum, so that cell would integrate a ramp rising from zero and count absorption in a band where the field carries no photon at all. Measured on the shipped optics, with $C_{\rm abs}$ from the Q table itself, it inflates the absorbed power of the smallest grains by 1.74% (falling to 0.05% for $a \geq 0.3\,\mu$m, which take little of their heating in the ultraviolet). Small grains are the ones the stochastic solver treats, so the error does not stay small: it moved the equilibrium/stochastic boundary by twenty radii and shifted the emergent SED by up to 13% at the far-infrared peak.

The extra grid point at $0.0912\,(1-10^{-4}) = 9.119088 \times 10^{-2}$ μm closes that cell, and with it the artifact: $1.74\% \to 0.006\%$. Its refractive index is interpolated rather than tabulated, which costs nothing — the astrodust dielectric function is smooth across 13.6 eV and has no edge there. The Lyman limit is a property of the illumination, and this is the same move the extension already makes at every one of the material's own edges: resolve a step with a close pair of points.

6.2 Where the optics come from in the extended band

- **Astrodust grains.** The T-matrix on the Draine & Hensley (2021) dielectric function — the same material *and* the same $b/a = 1.400$ oblate spheroid as the Q table, on the same random-orientation average, so the band continues the table's own calculation instead of substituting a different particle for it. This is the default; passing `euv_tmatrix=.false.` instead selects Bohren–Huffman Mie for the *volume-equivalent sphere*, much cheaper but an approximation. Both are described in §6.3. On the _euv table **neither is ever reached**, because no astrodust band forms below it and the flag has no effect; on the default 1129-wavelength table a `lam_min` below the Lyman limit reaches them.

- **PAH / carbonaceous.** Above 21.4 eV ($1/\lambda = 17.25$ μm^{-1}) the DL07 PAH cross section is zero by construction, so graphite is the *whole* carbonaceous absorption there. Below that energy the D16 turbostratic-graphite sphere table is used as before; above it the code switches to Mie on the D03 graphite dielectric functions (random orientation, $\frac{1}{3}$ parallel $+\frac{2}{3}$ perpendicular), because the D16 table's radius axis bottoms out at 10^{-3} μm while the astrodust size grid starts at 3.55×10^{-4} μm. In the Rayleigh limit $Q_{\rm abs} \propto a$, so clamping a smaller grain to the table's first radius overestimates its absorption by $10^{-3}\,\mu{\rm m}/a$ — up to 2.4× for the smallest PAH the size distribution

populates ($4.22 \times 10^{-4}\,\mu$m). *This branch is unreachable on the unextended grid*: its shortest wavelength gives $1/\lambda = 10.96 < 17.25$, so every previous result is unaffected.

- **DL07 model.** Its optics are Mie on the D03 dielectric functions at *every* wavelength, so nothing changes character across the band and the extension is a grid extension only.
- **Zubko model.** No extension, and none possible: its tabulated optics already span 10^{-3} to $10^{4}\,\mu$m, and those tables *are* the model definition, so there is no dielectric function to extend them with.

6.3 The two routes, and what the cheap one costs

Read this section as a description of a route that no longer has anything to compute for astrodust. It was written when the Q table stopped at the Lyman limit and every ionizing host needed a band solved on the fly. The table now covers that band, so the astrodust EUV path is unreachable — which is the point: no host has to link the T-matrix any more. The route is kept because it is what would compute a band if a future dielectric function reached further than the table built from it, and because the timings below are the measurement that justified moving the work into the table. In particular **the volume-equivalent-sphere substitution described here is used nowhere in the shipped astrodust products**: every wavelength of every one of them is read off the Q table, and no argument to `build_astrodust` can change that.

The Q table is a random-orientation T-matrix solution for an oblate spheroid of axial ratio $b/a = 1.4$. **By default the extended band is the same solution**, called directly (`spheroid_q` in `tmatrix/driver/spheroid_optics.f90`) on the same dielectric function, with the same convergence tolerance and the same regime bounds the table's own driver uses: the Rayleigh dipole limit below $x = 2\pi a_{\rm eff}/\lambda = 0.1$, the T-matrix between 0.1 and 50, the geometric-optics limit above 50. Nothing about the particle changes across the seam.

That solution lives in `sed/src/euv_astrodust_tmatrix.f90`, the one file of the library that reaches into `libtmatrix.a`, and it is reached through a registered procedure rather than a compiled dependency — which is what lets the rest of the library link without the T-matrix at all (§2.). A build that does not carry it answers `euv_tmatrix = .true.` with `status = 6`; it never falls back to the sphere below, because that is a different particle and the caller has to know.

That is not free. Measured on this machine (`gfortran` 13.1, `-O3`, 167 radii; the timings are the band alone, as `sed_init` reports them):

`lam_min`	$n_{\rm euv}$	band points	of which T-matrix	threads	time
0.0247968 μm (50 eV)	113	18 871	12 204	1	527.5 s
”				2	264.2 s
”				4	136.7 s
”				8	69.2 s
$1.001 \times 10^{-4}\,\mu$m (12.4 keV)	590	98 530	41 900	72	73.5 s

The second row is the floor `calc_kext.x astrodust euv` picks by default. Most of a deeper band is cheap, because past $x = 50$ it is the geometric-optics limit: going from 50 eV down to 12.4 keV multiplies the band points by 5.2 but

the T-matrix points by only 3.4.

The band is built with OpenMP over grain radii, with one T-matrix workspace for each thread (37 MiB at allocation, 80 MiB once its full-direct arrays are in place; nothing is allocated when there is no band to compute). That is the one thing to watch on a many-core host: peak resident memory measured 64 MB / 110 MB / 197 MB / 376 MB at 1 / 2 / 4 / 8 threads and 3.0 GB at 72, so cap `OMP_NUM_THREADS` if memory is tight. Each radius writes its own column and reads no other, so the result does not depend on the thread count or the schedule — the four runs above wrote byte-identical optics.

The cheap route. `euv_tmatrix=.false.` replaces the spheroid by its volume-equivalent *sphere* and solves it with Bohren–Huffman Mie, in a tenth of a second. The justification for that substitution is **not** the anomalous-diffraction argument $|m-1| \ll 1$: for this material $|m-1| = 0.77$ at the seam and still 0.56 at 20 eV. It rests instead on the two limits that actually apply:

- **Small grains are in the Rayleigh limit** ($x = 2\pi a/\lambda = 0.033$ at the seam for the smallest populated radius, and only 0.24 at 100 eV). There the ellipsoid polarizability is $\alpha_j = V(\epsilon-1)/\{4\pi[1+L_j(\epsilon-1)]\}$, so shape enters *only* through $L_j(\epsilon-1)$. And $|\epsilon-1|$ falls from 1.86 at 13.5 eV to 1.10 at 20 eV, 0.28 at 40 eV and 0.061 at 100 eV: the depolarization factors drop out and sphere and spheroid converge on the same $C_{\rm abs}$.
- **Large grains are in the geometric-optics limit**, where $C_{\rm ext} \to 2\times$(orientation-averaged projected area). By Cauchy's theorem that average is $S/4$, and for $b/a = 1.4$ the spheroid's $S/4$ exceeds $\pi a_{\rm eff}^2$ of its volume-equivalent sphere by 2.12%. Mie is therefore low by $1-1/1.0212 = 2.08\%$ in that limit. **The error converges to a constant, not to zero**, however far into the EUV one goes.

Consequently the error does *not* fall monotonically with photon energy. Only $a \lesssim 2\times10^{-3}\,\mu$m improves monotonically; $a \gtrsim 0.01\,\mu$m gets *worse* between 13.6 and 30 eV before recovering, and the largest sizes settle at a nearly energy-independent −1.9 to −2.0%. Over the whole band and the whole size range the error stays within about 2%; `sed/src/q_astrodust.f90` tabulates it size by size.

Measured against the default. Building the same model both ways with `lam_min` = 0.0247968 μm gives, for $\Delta C_{\rm abs}$ = (Mie − T matrix)/T matrix [%]:

a [μm]	13.8 eV	17.7	24.8	30.1	40.2	50
4.73×10^{-4}	−0.31	+0.71	−0.01	−0.08	−0.13	−0.02
1.00×10^{-3}	−0.14	+0.75	+0.05	−0.00	−0.03	−0.02
5.01×10^{-3}	−0.16	+0.75	−0.09	−0.17	−0.24	−0.23
1.00×10^{-2}	+0.30	+0.24	−0.60	−0.57	−0.37	−0.27
5.01×10^{-2}	−1.28	−1.47	−1.59	−1.58	−1.47	−1.27
1.00×10^{-1}	−1.62	−1.74	−1.85	−1.87	−1.86	−1.77
2.99×10^{-1}	−1.90	−1.96	−2.01	−2.02	(geometric optics)	

Over the $0.1 \leq x \leq 50$ block where the T-matrix itself answers, $\Delta C_{\rm abs}$ runs from −2.03 to +0.77% with a median of −0.70%, exactly the bound argued above. Where $x > 50$ both routes leave the T-matrix — the spheroid one for its geometric-optics limit, which is what the Q table uses there too — and the two disagree by −6 to −19%; that is Mie against a chord approximation, not against the T-matrix.

Seam continuity. Extrapolating the Q table's first two wavelengths log–log back to the last extended-band wavelength and comparing with what the band actually produced there, over all 167 radii:

route	median |step|	max |step|	RMS step
T-matrix (default)	0.012%	0.068%	0.027%
Mie sphere (`euv_tmatrix=.false.`)	1.167%	15.709%	6.186%

The default is continuous to a few hundredths of a percent, about a hundred times smoother than the sphere substitution. The asymmetry parameter behaves the same way: the table gives $\langle\cos\theta\rangle = 0$ wherever its own regime bounds put a radius in the Rayleigh or geometric-optics limit, and the default reproduces that exactly, whereas the sphere route steps from 0.90 to 0 across the seam for $a \gtrsim 1\,\mu$m.

The shipped extinction table. `data/astrodust/kext_astrodust_MW_euv.dat` now takes every wavelength from the Q table, the band included, so the extinction it serves and the emission the model computes are one calculation of one particle throughout — the property the paragraph below was written to establish, reached by extending the table rather than by solving a band. The comparison it records was made when the band was still computed on the fly: against the sphere route, the 1129 rows at $\lambda \geq 0.0912\,\mu$m were byte-identical, because those came from the Q table either way. Below the seam the sphere route was low in $C_{\rm abs}$ and $K_{\rm abs}$ by up to 3.6%, high in $C_{\rm sca}$ by up to 4.6% and in albedo by up to 5.1%, and wrong in $\langle\cos\theta\rangle$ by up to a factor of four at the shortest wavelengths, where the sphere keeps a forward-scattering asymmetry the spheroid's geometric-optics limit does not. $C_{\rm ext}$ moved least, by 0.72%. Regenerating costs about five minutes on 64 threads (`OMP_NUM_THREADS=64 ./calc_kext.x astrodust euv`). **The sphere route can no longer be selected here**: the table covers the band, so `build_astrodust` forms no band for `euv_tmatrix` to steer, and the comparison above cannot be reproduced without shortening the table first.

6.4 The single-photon ceiling comes from the field

The stochastic solvers set the top of a grain's enthalpy window from the energy of the hardest single photon it can absorb. That ceiling used to be the hard-coded Lyman limit, 13.6 eV, which an ionizing band makes wrong: a photon harder than the ceiling drives an excursion the window cannot hold. It is now

$$\max\big(13.6\,\mathrm{eV},\ hc/\lambda_{\rm hard}\big), \qquad \lambda_{\rm hard} = \min\big\{\lambda_i\ :\ J_\lambda(\lambda_i) > 10^{-12}\max_j J_\lambda(\lambda_j)\big\},$$

computed by `hardest_photon_energy` in `sed/src/radfield.f90` and called from all four places that need it — `sed_grain_loop` and `narrow_iterative` in `sed_astrodust.f90`, `qm_solve_grain` in `stoch_qm.f90`, and `calc_P` in `p_sub.f90`. Any units may be passed for J_λ, since only ratios within the array are used.

`calc_P` is the fourth because it does not merely bound a window: it integrates the upward transition rates over the photons that drive them, so both the cutoff and the limits of that integral are properties of the field. §6.5 is what it cost to have taken them from the grid instead.

The ceiling is a property of the field, not of the grid. That distinction is the whole point, and a coincidence hides it. A model's wavelength grid is the grid of its optics tables, which can reach far past the band a transported field occupies:

model	grid floor	implied photon energy
astrodust	$1.000 \times 10^{-4}\,\mu$m (T-matrix Q table)	12398 eV
DL07	$1.000 \times 10^{-4}\,\mu$m (the same table)	12398 eV
MRN	$1.000 \times 10^{-4}\,\mu$m (the same table)	12398 eV
Zubko	$1.000 \times 10^{-3}\,\mu$m (ZDA optics tables)	1239.8 eV

Every shipped model now depends on the distinction: the grid floors above sit 912, 912 and 91 times harder than a Mathis field's own, which is the Lyman limit, and the field is the one that is right — it carries no photon at all below $0.0912\,\mu$m, however far the optics table extends. Before the Q table was carried into the ionizing band, astrodust and DL07 could not tell the two apart: their grid floor *was* the Lyman limit ($13.595\,\text{eV} < 13.6\,\text{eV}$, so the maximum returned the constant whichever was measured), and Zubko was the only model that exercised the difference.

Taking it from the grid costs resolution, not physics. `umax` sets the upper edge of a *fixed* number of enthalpy bins, so a ceiling 91 times too high starts the solve with bins 91 times too coarse. The refinement loops do contract the window once the top bin is unpopulated, but the contraction is guarded (`umax > 1.02*umaxlo` and `umax > 1.01*ub(jcut)`) and capped at `MAX_ITER = 10`, so it does not return to where it began; expansion carries no such guard. Measured in a host code on the Zubko model, with no `lam_min` requested at all: dust temperature +0.0705 K median and positive in every cell, the ten brightest SED bins −0.9 to −1.9%, the emission image 1.9% median and 5.4% maximum, total emitted energy −0.08%. A systematic redistribution at nearly conserved energy is what a coarser bin grid produces, and `docs/EUV_EXTENSION_HOST_REGRESSION.md` records the measurement.

6.5 The photon that was not there

The ceiling above bounds a *window*. `calc_P` does something stronger: it integrates the upward transition rates over the photons that drive them. Until this release it took the limits of that integral from the grid, and the result was not a loss of resolution but a source of energy that does not exist.

The highest-bin term is the rate into the top enthalpy bin, driven by every photon more energetic than that bin's gap ΔH. Two things went wrong.

- **The integral started at `lambda(1)`**, the short end of the optics table, rather than at the shortest wavelength the field occupies. On the grid ending at the Lyman limit the two coincided; carried to $10^{-4}\,\mu$m they differ by 912, and a flat 51-point rule left between 1 and 8 samples inside the illuminated band.

- **The flux at the gap wavelength was read through a clamping interpolator.** `interp1` returns $y(1)$ for any abscissa below its range. The top bin is placed above the single-photon bound by construction, so $hc/\Delta H$ was always shorter than the grid, and the clamp answered every time with J_λ at the grid edge instead of zero — on a grid ending at the Lyman limit, the full Mathis intensity there. *Every top-bin transition was driven by a photon flux the field does not carry.* When ΔH exceeded the grid's whole range the same clamp made the loop run backwards, subtracting a rate rather than adding none.

Both limits now come from `hardest_photon_energy`, and the band is skipped when ΔH exceeds the hardest photon. The sample count follows the band's width (`NWAV_MIN`, `NWAV_MAX`, `DLNWAV_TARGET` in `p_sub.f90`) instead of being fixed, so the step is bounded whatever the band. Re-running the old grid with only the ghost removed and then only the sampling changed separates the two: the sampling moves nothing at all (0.0000%), which says the flat 51 points had already converged once the limits were right; the ghost carries the entire change.

All three models shipped at the time moved, and Zubko's move has nothing to do with the astrodust table — its grid has always begun at $10^{-3}\,\mu$m, so it carried the same defect independently. Against the previous products: astrodust up to 12.1% locally and −1.1 to +0.9% in integrated power, DL07 9.7% and −2.8%, Zubko 3.0% (6.0% with euv). Energy conservation, measured as emitted over absorbed power per H, improved from +0.236% to −0.057% for the astrodust S1 stage and held at −0.11% for S2.

Every SED this package produced before this fix contains photons that were not in the field, in the top enthalpy bin of every stochastically heated grain. Comparisons against those products have to account for it.

Why 10^{-12} of the peak, and not simply $J_\lambda > 0$. Exact zeros must be excluded — `J_Mathis` returns exactly zero below $0.0912\,\mu$m — but an exact positivity test is too fragile: a host that hands over denormal or roundoff-level residue in its unilluminated bins would push the ceiling back up by orders of magnitude. A component a fraction 10^{-12} below the peak of J_λ contributes at most that fraction of the photon absorption rate; allowing two decades for the run of $C_{\rm abs}$ across the illuminated band, the enthalpy states it could populate carry probability $\lesssim 10^{-10}$ of the peak, at or below the 10^{-13} tail level (`PMIN_UP`, `PMIN_UP_QM`) at which both solvers already truncate the window. Such a component cannot change a resolved excursion, and 10^{-12} still sits eighteen decades above the residue it is meant to reject.

The error is one-sided on purpose. Both refinement loops *expand* the window while the top bin is still populated, so an underestimated ceiling is corrected as the loop runs; the guarded contraction can stop short of where it began, so an overestimate is not corrected at all. `MAX_ITER = 10` bounds the correction in either direction. Overestimating is therefore the dangerous direction, which is why the threshold is set high enough to reject junk rather than as low as floating point allows.

`calc_sed.x zubko` exists to keep this path covered. The defect escaped `calc_sed.x astrodust` and `calc_sed.x dl07` because for those two models the grid floor and the field's short end are the same number: no driver in the tree ran the emission solvers on a model whose optics grid reaches past the illuminated band. `calc_sed.x zubko` (§7.3) closes that gap and is built by the default `make`. It prints the two candidates side by side, so the difference shows in the first lines of output:

```
$ ./calc_sed.x zubko # grid cut at the Lyman limit, Mathis field
 optics grid short end : 8.9984E-02 um -> 1.37784E+01 eV
 hardest photon in the field: 1.35946E+01 eV
 single-photon bound in use : 1.36000E+01 eV
$ ./calc_sed.x zubko euv # whole ZDA grid, still the Mathis field
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 1.35946E+01 eV
 single-photon bound in use : 1.36000E+01 eV
$ ./calc_sed.x zubko euv hardfield # ... and a hard component added to the FIELD
 optics grid short end : 1.0000E-03 um -> 1.23984E+03 eV
 hardest photon in the field: 3.70141E+02 eV
 single-photon bound in use : 3.70141E+02 eV
```

The middle run is the one that matters: the optics grid reaches 1.24 keV and the ceiling must still stay at 13.6 eV, because the field stops there. Only `hardfield`, which puts photons in that band, may move it — and it does, to the same 370 eV the field itself carries. `hardfield` implies `euv` for that reason: on the cut grid there is no wavelength to put those photons on.

A model built without `lam_min` is unaffected. For astrodust and DL07 the maximum selects the 13.6 eV constant on the unextended grid either way, so those results are unchanged to the last bit.

6.6 What else the extension does, and does not, change

- **The Planck integrals are anchored to the table range.** The internal log-λ integration grid for $\kappa_B(T,a)$ keeps its step and its sample points over the optics-table range and merely *prepends* points below it, so widening the model grid cannot coarsen the sampling of the range that carries the integrand. Measured: with a $1.001\times10^{-4}\,\mu$m floor, κ_B is unchanged to round-off over the thermal range ($\max|\Delta\kappa_B|/\kappa_B = 3.4\times10^{-16}$ for $T\le3000\,$K, and exactly zero below $\sim2400\,$K); $\kappa_{\rm CMB}$ is bit-identical. The only nonzero difference is 1.1×10^{-8} at the very top of a 5000 K grid, where the Wien tail genuinely reaches into the extended band — that is real extra emission, not a sampling artifact. Without the anchoring, spending a fixed budget of points on the widened interval would have moved κ_B by $\sim10^{-4}$ for a purely numerical reason.
- **Stochastic heating by hard photons is not solved by this change.** The extension fixes the photon-energy mapping and the absorbed-energy accounting. It does not address the finite enthalpy grid: upward transitions above the top of that grid are accumulated into the highest temperature bin, and for the smallest PAHs a 54 eV photon already exceeds $H(5000\,\mathrm{K})$. Nor does it add photoionization, electron energy loss, fragmentation or PAH destruction. Treat the extended band as extinction and absorbed-energy physics, not as a validated hard-EUV PAH *emission* model.

author-provided data	coverage	what it contains
Draine, `kext_albedo_WD_MW_3.1_60_D03` (DL07 / WD01)	10^{-4}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–12.4 keV	λ, albedo, $\langle\cos\theta\rangle$, $C_{\rm ext}$/H, $K_{\rm abs}$, $\langle\cos^2\theta\rangle$
HD23 astrodust release (`extinction.dat`, `scattering.dat`, `wav.dat`)	0.1–$3\times10^{4}\,\mu$m = 4.1×10^{-5}–**12.4 eV**	absorption and scattering $\tau/N_{\rm H}$ only. **No** $\langle\cos\theta\rangle$ **product exists anywhere in the release.**
Zubko ZDA optics tables	10^{-3}–$10^{4}\,\mu$m = 1.24×10^{-4} eV–**1.24 keV**	$Q_{\rm abs}$, $Q_{\rm sca}$, $Q_{\rm ext}$, g for each radius

Table 4: What the model authors published, and how far it reaches.

6.7 How far each model can be checked

The consequence is blunt: **the astrodust EUV band has no external reference at all**, because the HD23 release stops at 12.4 eV — exactly where the extension begins. What can be said is that the band is computed the same way the published part of the model is, from the same dielectric function and for the same particle (§6.3), and that it joins the table without a step. That is a consistency argument, not a validation.

The DL07 model *can* be checked, because Draine published the same model over the whole range. Comparing `data/dl07/kext_dl07_MW_euv.dat` with `kext_albedo_WD_MW_3.1_60_D03.all_2003` on the reference's own wavelengths (and counting only points where the reference grid is no finer than ours — elsewhere the comparison measures our grid, not our optics) gives, per decade in λ, mean $|\Delta C_{\rm ext}|/C_{\rm ext}$ of 0.056–0.86%, mean $|\Delta\,\text{albedo}| \le 0.0019$ and mean $|\Delta\langle\cos\theta\rangle| \le 0.00054$. `calc_kext.x dl07 euv` prints that table. The two residual features are worth naming: the excluded points cluster at the X-ray absorption edges (Fe K, Si K, Mg K, Fe L, O K, C K), which the reference brackets far more finely than our $\Delta\ln\lambda = 0.0116$ log grid can resolve; and the largest single deviation, 7.9% in the 1–10 μm decade, is confined to the 6.2, 7.6 and 7.9 μm PAH C–C features, where Draine's 2003 table uses a different vintage of the PAH cross sections.

7. Standalone programs

Every program here is named for the quantity it computes — `calc_sed.x` for the emission SED, `calc_kext.x` for the size-integrated transport optics, `calc_qtable.x` for the stored cross-section tables, `calc_enthalpy.x` for the enthalpy tables — so that none of them can be mistaken for the build tool. `make` in `SEDust/sed/` builds the default set; the rest have make targets of their own. All of them are run *from* `SEDust/sed/`, because every data path is relative to `../`.

They also share ONE command-line system (§7.2): one vocabulary, one parser, one refusal and one filename-tag rule, with each program declaring which axes it has a referent for.

7.1 `calc_kext.x` — size-integrated transport optics

Builds a model and calls `size_integrated_extinction` on it, then writes the result under `../data/`. Every model goes through the same two calls, and `dust_extinction` later serves these very files back to an RT host, so the files written here *are* the optics that host gets in memory.

```
./calc_kext.x astrodust [euv | <lam_min in um>]
./calc_kext.x dl07 [ld01] [euv | <lam_min in um>]
./calc_kext.x mrn [euv | <lam_min in um>]
./calc_kext.x zubko [formula | table] [zda | mie_d03] [euv]
./calc_kext.x themis [euv]
./calc_kext.x g18d [euv]
./calc_kext.x from_files <descriptor> [data_dir]
```

With no argument it prints its usage and stops. `euv` carries the grid down to whatever the model's own dielectric function covers; an explicit number requests a specific floor in μm. `zubko` defaults to `formula`; `table` marks the curve it writes, as `kext_zubko_BARE_GR_S_table[_euv].dat` and the `/kext_table` group, so that the two size-distribution routes – whose normalizations differ by 0.4–1.1% – cannot overwrite one another. `from_files` takes the descriptor path, and the data directory defaults to the descriptor's own directory. `ld01` asks for the DL07 model with the Li & Draine (2001) carbonaceous absorption in place of the Draine & Li (2007) one — everything else the same, so a ratio of the two curves isolates that one cross section — and lands beside it as `../data/dl07/kext_ld01_MW[_euv].dat`, with the matching `/kext_ld01` group inside the model's own product. The 2003 reference table `kext_albedo_WD_MW_3.1_60_D03.all_2003` was computed with those cross sections, which is why the point-by-point comparison this program prints at the end is the sharper test for that run. For astrodust the `euv` floor now falls inside the Q table, so `./calc_kext.x astrodust` and `./calc_kext.x astrodust euv` cover the same grid and write the same curve; only the field widths and the header differ.

Columns are

```
lambda[um]  albedo  <cos>  C_ext/H  C_abs/H  C_sca/H  K_abs
```

in μm and cm^2 per H nucleon, with $K_{\rm abs}$ in cm^2/g. The first four columns are in the order of Draine's `kext_albedo` tables, so a reader written for those files (for instance the `par%ion_dust_kext` reader of the MoCHII host, which takes four reals from each row and uses λ, albedo and $C_{\rm ext}$) parses these unchanged. Note the fifth column differs from Draine's: his is $K_{\rm abs}$ [cm^2/g], ours is $C_{\rm abs}$/H [cm^2/H], so no dust-mass normalization is folded into the first six columns. $K_{\rm abs} = (C_{\rm abs}/{\rm H})\,/\,(M_{\rm dust}/N_{\rm H})$ is written as a separate trailing column instead, on every product, because every model states its grain densities (§5.2). The header of each file records the $M_{\rm dust}/N_{\rm H}$ its $K_{\rm abs}$ column is normalized by and where that model's densities come from.

After writing, the program checks $C_{\rm ext} = C_{\rm abs} + C_{\rm sca}$ and the ranges of albedo and $\langle\cos\theta\rangle$, and — for astrodust and DL07 — prints the comparison against the author-provided data of Table 4.

These files are transport optics, not a dust model. They carry no size-resolved $C_{\rm abs}(\lambda, a)$, no grain enthalpy and no Planck integral, so stochastic heating and thermal emission cannot be computed from them. A host that needs emission

must build the model; the builder reads this curve back in, `dust_extinction` serves it on the model wavelength grid and `dust_emission` supplies the re-emitted spectrum, which is what keeps the absorbed power and the emission referring to one and the same grain.

7.2 The command-line system

Every program in `sed/` takes ONE vocabulary of run settings (`sed/src/sed_run_options.f90`), so that a word names the same physics whichever program reads it. The settings fall into independent *axes* (Table 5), and each program DECLARES which axes it has a referent for before it reads a single argument. Three things follow from that declaration:

- a word of a declared axis is read;
- a word of an axis the program did *not* declare is refused, naming the axis — `./calc_kext.x dl07 qm` answers `'qm' selects the stochastic-solver axis, which calc_kext has no referent for` — rather than being silently dropped, which is the failure this system exists to prevent. When the axis exists in the program but not in the model it was asked about, the model is named instead: `./calc_sed.x zubko c2` answers `. . . which calc_sed's zubko has no referent for`;
- anything else is the program's own positional argument (a DL07 submodel name, a wavelength floor, a descriptor path).

axis	words	programs that declare it
subject (first argument)	the model, or the product, the program works on	all but `calc_polext.x`
solver	`heuristic` (default), `draine`, `equil`, `qm`, `qm_dbcon`, `qm_stati`	`calc_sed.x`
grid	`euv`	`calc_sed.x`, `calc_kext.x`
field	`mathis_orig`, `logU=X`, `hardfield`	`calc_sed.x`
emission	`induced`, `photcut`	`calc_sed.x`
matrix size	`nstate=N`, `nisrf=N`	`calc_sed.x`
graphite of the ξ blend	`gra_d03_sphere`, `gra_d16_sphere`, `gra_d16_spheroid`	`calc_sed.x` (astrodust, dl07)
carbonaceous vintage	`dl07` (default), `ld01`	`calc_sed.x` and `calc_kext.x` (dl07)
Stage-1 enthalpy	`c2`	`calc_sed.x` (astrodust), `calc_enthalpy.x`
Zubko size distribution	`formula` (default), `table`	`calc_kext.x` (zubko)
Zubko optics set	`zda` (default), `mie_d03`	`calc_kext.x` (zubko)

Table 5: The run-setting axes, and which program has a referent for each. Any combination across axes is a valid run.

The three `qm` words name one solver and one cooling kernel between them: all three set `stoch_method = 'qm'`, and they set `qm_method` to `'dbdis'` (the default), `'dbcon'` and `'stati'` respectively (§4.2). They are values of one axis, not two settings, so `qm qm_stati` is the refusal below rather than a compound request.

Two kinds of combination are refused rather than resolved silently. Naming two values of one axis — two solvers, or two graphite sources — would leave the run doing one thing and its filename claiming another. So would naming a setting the chosen solver does not read: `nstate=` and `nisrf=` size the energy-space transition matrix, which only `qm`, `qm_dbcon` and `qm_stati` build, and `photcut` acts inside the bin sum of the heuristic narrowing solver, the only solver that reads it.

Every setting tags the output filename, so a variant run cannot overwrite the production one. Parsing and tagging are separate steps: reading an argument only records what was asked for, and the tag is assembled afterwards in one fixed order — grid, field extent, solver, graphite, optics set, enthalpy, field normalization, emission term, numerical sizes — so `induced qm` and `qm induced` write the same file. A default run carries no tag at all, which is what leaves the production filenames as they were. Each program prints the file it will write before it starts, so a mistyped setting is caught then rather than after the work.

Where a setting lands is a separate module (`sed_apply_options.f90`), because the answer is not the same for every program: one that computes an enthalpy table or a cross-section table links no solver and no radiation field, and would otherwise have to drag both in to get the shared command line.

7.3 `calc_sed.x` — emission SEDs

One program for the shipped models: `./calc_sed.x <model> [settings ...]`, the model name required and everything after it from the system of §7.2. It writes `output/sed_<model>[_<submodel>][_<tag>][_<stage>].dat`.

`astrodust` — the production SED. Solves the astrodust+PAH SED for a single Mathis-ISRF cell at $U = 1.585$ ($\log U = 0.20$, the HD23 best fit), on a 200-point log T grid from 2.7 to 5000 K. Reads the T-matrix Q table `../tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat` and `../data/release/size_distribution.dat`; writes `output/sed_astrodust[_<tag>]_S1.dat`, `...S2.dat` and `...PAH.dat`, each with λ [μm] and $\lambda I_\lambda / N_\mathrm{H}$. Optional arguments, in any order: the shared settings of §7.2, plus the two axes only this model has a referent for — the graphite of the PAH ξ blend and the Stage-1 enthalpy prefactor `c2`.

`gra_d03_sphere`, `gra_d16_sphere` and `gra_d16_spheroid` name the graphite optics the carbonaceous PAH-to-graphite ξ blend takes. `'d16_sphere'` is the production choice and the HD23 one — Draine 2016 turbostratic graphite, Maxwell-Garnett effective medium, on a sphere — and naming it outright changes nothing. `gra_d03_sphere` takes the Draine 2003 dielectric functions with Mie on a sphere, weighting $E \parallel c$ by $\frac{1}{3}$ and $E \perp c$ by $\frac{2}{3}$, which is the original DL07 choice; `gra_d16_spheroid` takes the same D16 material on Draine's $b/a = 1.4$ oblate spheroid table, random-orientation averaged. They tag their output `d03gra_`, `d16gra_` and `sphdgra_` (`output/sed_astrodust_d03gra_S1.dat` and so on). All three reach the PAH population only, so the S1 and S2 files they write are byte-identical to the production ones; what they move is the PAH sub-mm.

`dl07` — the DL07 reference SED. Solves the Draine & Li (2007) silicate + carbonaceous SED with the WD01 size distribution, at $U = 1$ unless `logU=X` asks for another intensity. `./calc_sed.x dl07 [model] [settings ...]`, where `model` is one of `mw31_00 ... mw31_60` (default `mw31_60`), `lmc2_00`, `lmc2_05`, `lmc2_10`, `smc`; an unknown name is rejected with the valid list and the settings this driver takes. Those are the shared settings of §7.2

plus the graphite of the ξ blend, which this model computes as well — its own choice being `'d03_sphere'`. Writes `output/sed_dl07_<submodel>[_<tag>].dat` with columns λ, total, silicate and carbonaceous $\lambda I_\lambda/N_\mathrm{H}$.
Naming a graphite source has one further effect here. The stored cross sections under `../data/dl07/` were computed once, with this model's own `'d03_sphere'` graphite, so reading them back would leave the request with no effect at all; a named graphite therefore solves every optic from the dielectric functions instead. That is what a comparison of two graphite choices needs anyway — both variants then take the same route — it costs about a second on the shipped grid, and `gra_d03_sphere` reproduces the stored-table default to every printed digit.

`zubko` — the Zubko/ZDA SED, and the wide-grid regression. Solves the Zubko, Dwek & Arendt (2004) BARE-GR-S SED — PAH + graphite + silicate, the ZDA size-distribution formula, the ZDA optics and calorimetry tables — for a Mathis ISRF at $U = 1$, and writes `output/sed_zubko[_<tag>].dat` with columns λ, total, PAH, graphite and silicate $\lambda I_\lambda/N_\mathrm{H}$.

```
./calc_sed.x zubko [settings ...]
```

The settings are the shared vocabulary of §7.2; there is no graphite axis here, because this model is its distributed optics tables and computes no ξ blend of its own. The solver defaults to `heuristic`, the model default. `euv` builds the model on the whole ZDA optics range, $10^{-3}\,\mu$m (1.24 keV) upward, instead of cutting it at the Lyman limit where the Mathis field stops. `hardfield` then replaces the field below the Lyman limit — where the Mathis field is identically zero — by a diluted 10^5 K blackbody, so that the field really does carry photons above 13.6 eV; the dilution is chosen so that band deposits roughly as much power as the whole Mathis field — enough for the hard photons to matter, little enough that the small grains still heat stochastically. It implies `euv`, a cut grid having no wavelength to put those photons on.

This is the emission counterpart of `./calc_kext.x zubko`, which exercises only the extinction size integral. It exists because Zubko is the one shipped model whose optics grid ($10^{-3}\,\mu$m) reaches far past the band a transported field occupies ($0.0912\,\mu$m for the Mathis ISRF), so anything in the stochastic-heating path that reads the grid where it should read the field shows up here and nowhere else. Running it with and without `euv` is the regression of §6.4.

`themis` and `g18d` — two published models read from their DustEM input files. Solve the THEMIS model (Jones et al. 2013 for the carbon grains, Köhler et al. 2014 for the silicates) and Model D of Guillet et al. (2018) for a Mathis ISRF at $U = 1$, and write `output/sed_themis[_<tag>].dat` and `output/sed_g18d[_<tag>].dat` with columns λ, total, and then one column per grain population in the order of the model's `GRAIN` file — the layout of DustEM's own `SED_*.RES`.

```
./calc_sed.x themis [settings ...]
./calc_sed.x g18d [settings ...]
```

Both models *are* their DustEM files: the size-distribution shape, mass per H, density and radius range come from `GRAIN_*.DAT`, the cross sections from `oprop/Q_<gtype>.DAT` on the common grid `oprop/LAMBDA.DAT`, and the enthalpy from the heat capacity tabulated in `hcap/C_<gtype>.DAT`. Nothing is recomputed from a dielectric function, so there is no graphite axis and no `lam_min` extension here. They are peers of the other four in every other respect: `build_dust(m, 'themis', data_dir)` and `build_dust(m, 'g18d', data_dir)` build them by name, `dust_lib`

exports `build_dustem` beside the other builders, `./calc_qtable.x themis` and `./calc_qtable.x g18d` write their `data/<model>/sedust_<model>.h5` products (§12.4), and `dust_extinction` serves each model's own curve without the host naming a file. Both are *scalar*: what the distribution publishes for them is orientation-averaged, so they carry no polarized optics. Their grid starts at 0.04 μm, shortward of the Lyman limit, so `euv` keeps the whole of it and its absence cuts it there — the same convention as `zubko`, and for the same reason. Hensley & Draine (2023), §6.2.2 and their Fig. 16, compare the astrodust model against exactly these two.

`./calc_kext.x themis euv` is checked against DustEM's own published extinction for the same files, `../data/themis/reference/EXT_J13.RES`, and `g18d` against a DustEM run on its own files, `../data/g18d/reference/EXT_G17_ModelD.RES` (`data/g18d/where.txt` records how that one was produced). `./test_dustem_extinction.x` builds both models and compares the total extinction and every population's absorption and scattering against them, and passes at 5×10^{-7} relative, the reference files' own print precision. It says how many wavelengths each column was compared over: a column the reference holds at zero – the G18D PAH scattering, DustEM's table for it carrying none – is checked for being zero on both sides instead of being turned into a ratio.

What is and is not shared with the reference. The comparison above is a comparison of two *solvers*, because everything else on the two sides is the same file or the same formula:

input	this work	DustEM reference
size distribution	`GRAIN_J13.DAT`, its own formula	same file, same formula
optics	`oprop/Q_<gtype>.DAT`	same files
asymmetry	`oprop/G_<gtype>.DAT`	same files
calorimetry	`hcap/C_<gtype>.DAT`	same files
wavelength grid	`oprop/LAMBDA.DAT`, 800 points	same file
radius grid	`nsize` per `GRAIN` line	same construction
size integral	trapezoid in $\ln a$	`XINTEG2`, trapezoid
proton mass	$1.67262158 \times 10^{-24}$ g	DustEM's `xmp`
radiation field	`mathis_orig`, analytic	`ISRF.DAT`, 200-point table
U	1	`G0` = 1
stochastic solver	Guhathakurta–Draine matrix (§4.2)	Désert et al. (1986), continuous cooling

Only the last row differs, and the radiation field differs in form rather than in content: the analytic Mathis field and DustEM's 200-point tabulation of it agree to 0.3–1% over 0.1–8 μm.

Extinction, which the solver never touches, therefore agrees to 5×10^{-7} at every one of the 800 wavelengths, not merely in the mean. Emission, which is the solver, agrees to a median −2.6% over 1–3000 μm, with band-integrated powers (ours over reference) of 0.983 in the near-infrared, 0.972 across the PAH features, 0.961 over the far-infrared peak and 0.9995 in the sub-millimetre; the largest single departure is −8.0% at 67 μm, on the warm shoulder that the smallest stochastically heated grains build, which is where two stochastic-heating methods are expected to part company. The

comparison is plotted in docs/figs/dustem_themis_comparison.pdf, made by docs/make_dustem_figs.py.

g18d carries **no scattering asymmetry**. The DustEM distribution ships no G_<gtype>.DAT for its two large populations, which hold 90% of the dust mass, so <cos> is not defined for that model: m%gsca_complete comes back .false., both drivers say so on stdout, and size_integrated_extinction returns $\langle\cos\theta\rangle = 0$ at every wavelength rather than averaging the one population that does carry a g over the scattering of all three. $C_{\rm ext}$, $C_{\rm abs}$, $C_{\rm sca}$ and the albedo are unaffected.

7.4 calc_enthalpy.x — enthalpy tables (diagnostic)

Tabulates $U(T, a_{\rm eff})$ for the two astrodust enthalpy stages and writes output/enthalpy_S1.dat and output/enthalpy_S2.dat (201 log-spaced temperatures from 2.7 to 5000 K × the 169 radii of ../data/astrodust/DH21_aeff, T outer and a inner). Its one axis is c2, the Stage-1 density-corrected prefactor, which tags its table (output/enthalpy_S1_c2.dat) and writes no Stage-2 file, Stage 2 not reading that setting. **It is not required to compute an SED**: the solver calls the closed-form Debye sums directly in its inner loop. Use it to compare the two stages, to plot $U(T)$, or to spot-check against published DL01 values.

8. The model object and accessors

```
type :: dust_model_t
   character(len=32) :: name
   integer :: NA, NLAM, NT
   real(wp), allocatable :: lam(:) ! (NLAM) [um]
   real(wp), allocatable :: aeff(:) ! (NA) [um]
   real(wp), allocatable :: T_first(:) ! (NT) [K]
   real(wp), allocatable :: log_T_first(:) ! (NT)
   type(grain_pop_t), allocatable :: pops(:)
   integer :: n_channel
   character(len=CHANNEL_NAME_LEN), allocatable :: channel_name(:)
   logical :: use_induced_emission ! default .false.
   logical :: gsca_complete ! default .true.
   character(len=16) :: stoch_method ! default 'heuristic'
   logical :: verbose ! default .false.
end type
```

Each grain_pop_t in m%pops carries its own aeff, dn, Cabs, Csca, gsca, kappB, H and kappCMB (plus cached logs), and the scalar rho_bulk [g/cm^3], the solid density of the material that population is made of. A model whose components have different size grids — Zubko is the case in the tree — is represented by populations of different lengths, which is why the solver takes the size count from size(pops(ip)%dn) rather than from m%NA.

channel_name. CHANNEL_NAME_LEN (129, in dust_model_mod) is wide enough for any name the library can build, so no channel name is ever cut. A model defined by a DustEM GRAIN_*.DAT names each channel after its grain population, and that name is the one the product's /qtable group is written under: normally the gtype alone, but a gtype appearing twice in one model carries its size-distribution keyword as well. THEMIS is the case that needs it, listing CM20 both as the power law with exponential decay and as the log-normal over different radius ranges, which become the channels CM20_plaw-ed and CM20_logn. The longest name any shipped model uses is the 20 characters of Model D's aSil2001BE6pctG_0.4x.

gsca_complete. Whether every population that scatters also carries a scattering asymmetry. It is .true. for every model whose optics come from a Mie or T-matrix calculation, which yields g alongside $Q_{\rm sca}$, and that is its default. A model defined by published tables can fail it: the DustEM distribution of Guillet et al. (2018) Model D ships no G_ file for its two large populations, so those populations leave gsca unallocated and size_integrated_extinction returns $\langle\cos\theta\rangle = 0$ for the whole model. A partial sum over the populations that do carry g, divided by the scattering of all of them, is the asymmetry of nothing and reads as a small g rather than as a missing one, which is why it is not written.

rho_bulk. It is what turns dn into a mass, and the only thing dust_mass_per_H (§5.2) reads besides the size distribution. For a porous grain it is the density already reduced by the porosity, i.e. mass over the *grain* volume $\frac{4}{3}\pi a_{\rm eff}^3$, not over the solid volume inside it — which is why the astrodust value is $\rho_{\rm Ad} = 2.74$ rather than the 3.425 of the solid it is made of. Leaving it at its default 0 declares that the model states no density, and that population then contributes no mass.

Diagnostics. m%verbose (default .false.) keeps the library solve path silent; set it .true. to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in dust_lib):

```
n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic, character(len=CHANNEL_NAME_LEN)
```

9. The generic file loader

build_from_files reads a small text *descriptor* and assembles a model from data files, with no code changes. One pop: line per population:

```
# data/zubko/zubko_descriptor.txt
name = zubko_bare_gr_s
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat ZDA_BARE_GR_S_SzDist_PAH.dat Graphitic_Calorimetry_1000.dat 0.0
pop: gra 2 Gra_121_1201.dat ZDA_BARE_GR_S_SzDist_Gra.dat Graphitic_Calorimetry_1000.dat 0.0
```

```
pop: sil 3 suvSil_121_1201.dat ZDA_BARE_GR_S_SzDist_Sil.dat Silicate_Calorimetry_1000.dat 0.0
```

- `grain_type`: `'sil'`, `'pah'`, or `'gra'` (selects the heat-capacity/mode model used by the `'qm'` solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- `channel`: integer output channel ($1 \ldots$); populations sharing a channel are summed.
- `optics`: a ZDA Q-table ($Q_{\rm abs}$ per radius and wavelength); $C_{\rm abs} = Q_{\rm abs}\pi a^2$.
- `dnda_table`: a 2-column table a [μm] and $\mathrm{d}n/\mathrm{d}a$ [cm^{-1} H^{-1}].
- `calorimetry`: a specific-enthalpy table $u(T)$ [erg/g]; $H(T,a) = u(T)\,\rho\,\frac{4\pi}{3}a^3$.
- `rho`: grain density [g cm^{-3}]; `0.0` uses the value in the optics-file header.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

10. Linking into a 3D RT code

Compile your RT source against the one archive, with the `.mod` search path pointing at `SEDust/sed/`:

```
gfortran -I<sed> my_rt.f90 -L<sed> -lsedust -fopenmp -o my_rt.x
```

The line is the same whether or not the T-matrix was built in (§2.): `WITH_TMATRIX=1` puts the T-matrix objects into `libsedust.a` itself, so the host never names a second archive. The archive needs no OpenMP runtime of its own — the threading is on the SEDust side — so `-fopenmp` is required only because `libsedust.a` uses it.
Only an astrodust EUV band (§6.3) ever reaches the T-matrix — and the shipped Q table leaves no such band to compute — and it reaches it through a procedure the host registers rather than a compiled dependency, which is why the plain archive links without one. A host with its own spheroid solver can register that instead: `sed_register_euv_band_optics` takes any procedure matching the abstract interface `euv_band_optics_i` (both re-exported by `dust_lib`), and `sed_forget_euv_band_optics` undoes it.
A complete minimal example is provided in `SEDust/sed/rt_example/use_dustlib.f90`: it builds a model, computes emission for one field, and prints the SED peak. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default `'heuristic'` solver is serial per call, which suits an outer cell-level parallel loop.

11. Units and conventions

- Wavelengths `lam` are in μm throughout; cross sections in cm^2.
- `J_lam` is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_{\rm H}$ is then in erg s^{-1} cm^{-2} sr^{-1} H^{-1}.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

12. The HDF5 optics products

Every model's optics ship as one HDF5 file,

`data/astrodust/sedust_astrodust.h5` `data/dl07/sedust_dl07.h5` `data/zubko/sedust_zubko.h5`

holding that model's wavelength axis, its $(\lambda, a_{\rm eff})$ cross-section tables and its size-integrated extinction curve. One file per model, because the models share no grid, no radius grid and no component list, and because a host ships only the models it runs. This is the primary form; the text products (§13.) are still written beside it and stay readable with an eye and diffable.

12.1 One wavelength axis, and what `include_euv` means

Each file carries *one* wavelength axis — the widest the model has — and records `i_lyman`, the index of the *last* wavelength at or shortward of the Lyman limit, 0.0912 μm:

```
/ attrs: model, generator, layout
/grid/lambda (n_lam) [um], strictly increasing
/grid attrs: i_lyman, lambda_lyman_um, i_lyman_meaning
/kext/{albedo,g,C_ext,C_abs,C_sca,K_abs} (n_lam)
/kext attrs: M_dust_per_H, size_dist_file, text_product, method
/qtable/<comp>/a_eff (n_a) [um]
/qtable/<comp>/{Q_ext,Q_abs,Q_sca,g} (n_lam, n_a) Fortran = (n_a, n_lam) in h5py
/qtable/<comp>/regime astrodust only: 0 T-matrix, 10 Rayleigh, 20 geometric optics
/qtable/<comp> attrs: rho_bulk_g_cm3, method, source, scatters
```

A reader given `include_euv = .false.` — the default — returns `lambda(i_lyman:)` and the same slice of every wavelength-indexed array; `.true.` returns the whole axis. The narrow product is therefore a *view* of the wide one rather than a second file to keep in step with it: the two cannot disagree, and there is nothing to regenerate twice. That is what replaces the paired `*_euv.dat` text products. The cut is an HDF5 hyperslab, so a non-ionizing host reads only the rows it keeps.

Measured i_lyman: astrodust 634 of 1762, DL07 695 of 1823, Zubko 336 of 1201 — giving 1129, 1129 and 866 non-ionizing wavelengths, the lengths of the corresponding text products.

The rule is the *last* node at or below the limit, not the first at or above it, because the non-ionizing view is a table an RT host interpolates onto its own grid: it has to *cover* the band the host transports. The astrodust and DL07 axes carry $0.0912\,\mu$m exactly, so for them the two rules return the same index. The ZDA grid does not, and its first node above the limit sits 1.2×10^{-5} of itself inside it — just past dust_extinction's edge allowance, so the same call that was served for two models used to be refused for the third. One extra node of the model's own table is not extrapolation; widening the allowance instead would have weakened the guard for every model at every wavelength.

model	components	n_a
astrodust	astrodust, pah_neu, pah_ion	169
dl07	sil, gra, pah_neu, pah_ion	84
mrn	sil, gra	70
zubko	sil, gra, pah	121, 121, 28
	sil_mie_d03, gra_mie_d03, pah_mie_d03	121, 121, 28
themis	CM20_plaw-ed, CM20_logn, aPyM5, aOlM5	25 each
g18d	PAH0_MC10, amCBE_0.3333x, aSil2001BE6pctG_0.4x	10, 25, 25

Table 6: Grain populations each product carries. Astrodust does *not* split into silicate and graphite: one composite grain plus PAHs is the premise of HD23, and the 'sil' grain-type tag its populations carry internally is a legacy label for the enthalpy route, not a claim about the material. A population whose cross sections are an absorption prescription rather than a Mie solution carries Q_abs alone, and its scatters attribute says so, so that an absent scattering term cannot be mistaken for a computed zero. Zubko carries *two* sets on one axis — see §12.3. The two DustEM-defined models name their groups by grain population rather than by material, because that is what their GRAIN_*.DAT defines; the THEMIS file names CM20 twice, with different radius ranges, so its two carbon populations are told apart by the size-distribution keyword they carry. Both large g18d populations have *no* g dataset — see §12.4.

12.2 What is stored in what precision

The products hold two classes of dataset, told apart by what a dataset *is* rather than by how large it is:

class	file type	datasets
values	64-bit float	the computed Q_ext, Q_abs, Q_sca, g, albedo, C_ext, C_abs, C_sca, K_abs, and the coordinate axes lambda, a_eff they are tabulated on
codes	8-bit signed integer	regime

Why the values are 64 bits. A number read back out of a product is the number that was computed. Nothing is lost to storage, so a comparison against a product never has to leave room for what the file type discarded, and the product

side of such a check is not the side that has to be given slack. An axis makes the same requirement sharper still: it has to stay strictly ascending however finely it is spaced. The astrodust wavelength grid resolves each X-ray absorption edge with a pair of points 6.7×10^{-7} apart in relative wavelength, and a grid that stopped being strictly ascending has broken this code once already, when a six-digit text wavelength column made `load_kext_table` reject the file.

Why `regime` is one byte. It takes three distinct values — 0 for the T-matrix, 10 for the Rayleigh dipole limit, 20 for geometric optics — and nothing between them. Eight bytes would carry them faithfully, but would at the same time say the quantity is continuous when it is not.

The reader interface is unchanged. Nothing on the reading side ever depended on any of this: the memory type on both write and read is `H5T_NATIVE_DOUBLE`, HDF5 converts, and a caller keeps passing and receiving `real(real64)`. `./test_h5_storage_policy.x` reads the file type of every dataset of every product in the tree and checks it against these two classes, which is what keeps a type from silently changing later. A product written before the code policy is brought onto it in place — no recomputation, every value, chunk, filter and attribute carried over — by `python3 pyutil/migrate_h5_regime_int8.py`.
On disk the six shipped products come to 45.0 MB: astrodust 9.31, dl07 9.77, mrn 6.47, zubko 16.31, themis 2.16 and g18d 0.94 MB.

12.3 Zubko carries two sets of optics

The ZDA BARE-GR-S model is defined by Zubko, Dwek & Arendt (2004), and the paper is where it would have to be read from. What this tree ships beside it are the three grain-input files the Camps et al. (2015) benchmark distributes —

```
data/zubko/suvSil_121_1201.dat data/zubko/Gra_121_1201.dat
data/zubko/PAH_28_1201_neu.dat
```

byte-identical to `SHG_Benchmark/DustModel/GrainInputs/` — so that the participating codes all run one agreed reproduction. Their headers record how they were made: Zubko's multilayer-sphere code (1997–2002), converted by K. Misselt in 2002, for the silicate and graphite; a 2009 implementation by Misselt of Li & Draine (2001) / Draine & Li (2007) for the PAHs.
Those files are the **default**. They are stored in `/qtable` of the product as they stand — `check_build_dust.x` reads both sides and requires an exact match — so a run against that benchmark measures the stochastic-heating solver and not an optics difference. This tree's own recomputation, Mie on the Draine (2003) optical constants, sits beside them under names suffixed `_mie_d03`, and each set has its own extinction curve so that what `dust_extinction` serves is the size integral of the very optics the model was built on:

```
/qtable/{sil,gra,pah} the benchmark's tables <- default
/kext ... and their size integral
/qtable/{sil,gra,pah}_mie_d03 Mie on the D03 constants
/kext_mie_d03 ... and their size integral
```

```
call build_dust(m, 'zubko', '../data', status=st) ! default
call build_dust(m, 'zubko', '../data', status=st, zubko_optics='mie_d03')
```

`build_zubko` takes the same choice as `optics`, and `calc_kext.x zubko [euv] [zda|mie_d03]` writes the matching curve. A name that is neither is refused (`build_zubko` status 8, `build_dust` status 92) rather than silently defaulted.

The recomputation reproduces the distributed silicate to a mean relative 2.4×10^{-6} and the graphite to 0.45% in $Q_{\rm sca}$ and 8% in $Q_{\rm abs}$. Its PAH component is a different implementation of the same LD01/DL07 prescription; measured against `Gra_121_1201.dat` cell by cell, the distributed PAH file holds graphite $Q_{\rm sca}$ and $\langle\cos\theta\rangle$ at every size and all 1201 wavelengths, and graphite $Q_{\rm abs}$ shortward of 0.0585 μm (21.2 eV, the DL07 PAH-to-graphite transition), so the ZDA PAHs scatter as the graphite sphere of their own radius. The recomputation now does the same. Through one builder and one size distribution the two sets differ by at most 0.17% in $C_{\rm sca}$ and 8.4% in $C_{\rm abs}$, the latter in the far-infrared where the graphite $Q_{\rm abs}$ discrepancy sits.

12.4 THEMIS and G18D: the published tables, on the model's own radii

These two models *are* their DustEM `Q_`/`G_` tables, and there is no dielectric function behind them for this tree to solve anything from. Their products therefore hold no recomputation. What they hold is the one step between the distribution's files and the numbers the size integral uses: `/grid` is `oprop/LAMBDA.DAT`, the 800-point DustEM wavelength grid every population shares, and each `/qtable` group is that population's $Q_{\rm abs}$, $Q_{\rm sca}$ and (where one is published) $\langle\cos\theta\rangle$ put on the *model's* radii — the `nsize` points the `GRAIN` line spaces evenly in $\ln a$ between its `amin` and `amax` — interpolated linearly in radius, which is what DustEM does with the same tables. $Q_{\rm ext}$ is written as the sum of the other two, the tables carrying absorption and scattering and not extinction.

That interpolation is in radius alone, so cutting the wavelength axis and interpolating commute: a model built from the product and one built from the text tables are the same numbers to rounding, not to the seven digits a text product would carry. `check_build_dust.x themis` and `... g18d` build both ways and require 10^{-10} relative agreement in $C_{\rm ext}$, $C_{\rm abs}$, $C_{\rm sca}$ and $\langle\cos\theta\rangle$. Measured, the size integrals and $M_{\rm dust}/N_{\rm H}$ agree exactly, at 0.

The curve `dust_extinction` serves does not, and the cause is not storage. That comparison reads `/kext` from the product on one side and the text `kext_<model>_euv.dat` table on the other, and those two shipped files are out of step with each other: the distributed `kext_themis_euv.dat` and `kext_g18d_euv.dat` predate the `/kext` group beside them. Recomputing both from the same run, as `calc_kext.x themis euv` and `... g18d euv` do, brings them to 4.5×10^{-13} of each other, the printed precision of the text file; against the distributed text files the same `/kext` sits 1.9×10^{-8} (THEMIS) and 2.6×10^{-8} (G18D) away. So four of these checks are reported over tolerance, at values that have nothing to do with how the product stores its numbers, and the text curves are what wants regenerating.

Where a g is missing, nothing is written. The DustEM distribution ships no `G_amCBE_0.3333x.DAT` and no `G_aSil2001BE6pctG_0.4x.DAT`, so those two `g18d` groups carry `Q_ext`, `Q_abs`, `Q_sca` and **no g dataset at all**. The absence is the record: a zero written there would be a measurement, and a model built from the product would then report an asymmetry the model does not have. `build_dustem` reads the absence back and reaches the same `gsca_complete`

= .false. it reaches from the text tables, which check_build_dust.x also checks, population by population.

12.5 build_dust: one entry point

```
use dust_lib, only: dust_model_t, build_dust
call build_dust(m, 'zubko', '../data', include_euv = .false., status = st)
```

One call for 'astrodust', 'dl07', 'mrn', 'zubko', 'themis', 'g18d' and 'from_files', built from one directory and one flag:

```
call build_dust(m, model, data_dir [, NT, T_lo, T_hi] [, include_euv] [, status] &
               [, message] [, lam_min] [, kext_path] [, sd_index] [, u_isrf] &
               [, zubko_optics] &
               [, config_path] [, astrodust_index_path] &
               [, euv_tmatrix])
```

NT, T_lo and T_hi are the internal temperature grid and are optional, defaulting to 200, 2.7, 5000 — the values the rest of this manual quotes as typical. They are what the EMISSION is solved on: the enthalpy $H(T,a)$ and the Planck-averaged opacity $\kappa_B(T,a)$ are tabulated on that grid, calc_Teq interpolates $T_{\rm eq}$ in it, and the stochastic solver's window is CLAMPED to $[T_{\rm lo}, T_{\rm hi}]$ rather than extrapolated past it — so the range must bracket the temperatures the field produces, and NT sets how finely $T_{\rm eq}$ is resolved. The extinction reads none of it, so a host that only wants dust_extinction can leave all three out.

data_dir is the SEDust data directory. The optics come from data_dir/<model>/sedust_<model>.h5 — the wavelength axis, the cross-section tables and the extinction curve alike. What is *not* optics still comes from beside it: the size distribution (data_dir/release/size_distribution.dat, or the ZDA config) and the calorimetry.

include_euv and lam_min each mean *one* thing, for every model. include_euv selects the view: .false. cuts the model's own axis at i_lyman, so the grid a host gets covers the Lyman limit whatever model it named. lam_min states the shortest wavelength the model must *cover*, and never truncates: astrodust, DL07 and MRN meet it by extending on the dielectric function their optics come from, while Zubko, THEMIS, G18D and a file-defined model, which have no dielectric function behind their tables, meet it when those tables already reach and *refuse* it when they do not.

Both used to be model-dependent, and a host with one call and one physically motivated floor met the difference: include_euv was an index cut for two models and a wavelength cut for the third, and lam_min extended two models and truncated the other two.

The builders of §3. are unchanged and still exported, so a host that names its own files keeps working. Two of them gained an optional argument that build_dust uses: include_euv on build_astrodust (which axis to take out of an HDF5 Q table) and lam_axis on build_dl07 (the model's wavelength axis outright). The second is why a host running DL07 alone needs no astrodust product: that model takes only a *grid* from a Q table, so build_dust hands it /grid/lambda out of sedust_dl07.h5 and no Q table is opened.

12.6 The existing builders take an HDF5 path too

Every place that names an optics file dispatches on the `.h5` suffix, so no caller needs a second flag to say which form it meant:

```
call build_astrodust(m, '../data/astrodust/sedust_astrodust.h5', NT, T_lo, T_hi, &
                     status=st, include_euv=.false.)
```

The same holds for `kext_path`. And with `kext_path` absent, a builder now tries `../data/<model>/sedust_<model>.h5` *first* and falls back to the text default listed in Table 7, and reads it the same way. `/kext` is read on the *whole* wavelength axis, never the non-ionizing slice: the curve is interpolated onto the model grid, so the widest one covers every grid the model can be built on. This is what retires the pairing of a narrow model with an `_euv`-only table, which used to fail at `dust_extinction` rather than at the build.

12.7 Generating them

```
cd sed
./calc_qtable.x # /grid and /qtable, every model
./calc_qtable.x themis # ... or one model on its own
./calc_kext.x <model> euv # /kext, for each of the six
```

The order matters. `calc_qtable.x` lays down the wavelength axis, so it creates the file with `H5F_ACC_TRUNC` and `/kext` goes with it; the `euv` run of `calc_kext.x` puts it back. That is the only order in which the two can be consistent, since a curve integrated on the previous axis would be filed against wavelengths it was not computed at. `calc_qtable.x` says so on stdout, and `calc_kext.x` writes `/kext` only if the grid it integrated on equals `/grid/lambda` to 10^{-12}.

12.8 Reading them from Python

`pyutil/sedust_h5.py` is the counterpart of the Fortran reader: same file, same cut, same numbers.

```
from pyutil.sedust_h5 import read_grid, read_kext, read_qtable, describe

k = read_kext('data/zubko/sedust_zubko.h5') # non-ionizing, 866 points
q = read_qtable('data/dl07/sedust_dl07.h5', 'sil', include_euv=True)
describe('data/astrodust/sedust_astrodust.h5') # or: python -m pyutil.sedust_h5 <file>
```

The cut comes from the file's own `i_lyman`, never from a wavelength comparison of the reader's own, so Fortran and Python slice at the same node. Cross sections come back `(n_a, n_lam)`: Fortran declares them `(n_lam, n_a)` and HDF5 stores them in that order, so each grain radius is one contiguous spectrum and the wavelength is the last axis. Nothing is computed on read — a value the file does not carry comes back `None`, never a zero that could be mistaken for a computed one.

12.9 Building without HDF5

`make HDF5=0` (and `HDF5=0 ./build_lib.sh`) compiles the HDF5 paths out. Every entry point of the layer then reports "not available" through its ok argument and the callers fall through to the text products, so no consumer carries an `#ifdef` and nothing has to be configured away. The same happens at run time in a tree that simply has no products yet. A default archive references the HDF5 libraries, so a host linking it must put `-lsedust` *before* them: a static archive resolves left to right, and it is `libsedust.a` that needs those symbols.

13. Data files

Model data lives under `SEDust/data/`, in **one directory per model** plus what the models share:

- `astrodust/`, `dl07/`, `zubko/` — everything that model owns: its HDF5 product `sedust_<model>.h5` (§12.), the same optics as text (`q_*.dat`, `kext_*.dat`), and, where the model IS a set of files, its definition. For astrodust that includes the T-matrix Q tables and the orientation-resolved DH21 table with its `DH21_wave` / `DH21_aeff` axes; for Zubko, the ZDA config/formula, the tabulated size distributions, the ZDA Q-tables and the calorimetry tables from the Camps et al. 2015 benchmark (a ready descriptor is `zubko_descriptor.txt`).
- `dielectric/` — shared material data: the DH21 astrodust index `index_DH21Ad_P0.20_0.00_1.400` (which the astrodust Q table was computed from, and which fixes how far `lam_min` may reach), the D03 astrosilicate and graphite indices, the D16 turbostratic-graphite tables, and the PAH cross sections. A dielectric function belongs to no one model — DL07 and Zubko read the same D03 astrosilicate — so it does not sit inside a model directory.
- `release/` — the published reference tables and the HD23 size distribution (`size_distribution.dat`), which astrodust and DL07 share.

The rule is that a host ships `data/<model>/` for each model it runs, plus `dielectric/` and `release/`. `build_dust` takes `data_dir` and resolves the rest, so nothing names a product file.

13.1 Transport-optics products

`calc_kext.x` (§7.) writes these into `SEDust/data/`. The last column is the point that matters when choosing one: **nothing here is a free choice — each floor is fixed by the data the model is made of**, and a shorter floor is refused rather than served with a refractive index frozen at its table boundary.

file	rows	λ range [μm]	what sets the short-wavelength floor
`kext_astrodust_MW.dat`	1129	0.0912–39810	the non-EUV Q-table grid
`kext_astrodust_MW_euv.dat`†	1762	10^{-4}–39810	the _euv Q-table grid: the DH21 astrodust dielectric function stops at 1.00003×10^{-4}, which that table already reaches, so euv prepends nothing beyond it
`kext_dl07_MW.dat`	1129	0.0912–39810	the non-EUV Q-table grid
`kext_dl07_MW_euv.dat`†	1823	6.205×10^{-5}–39810	the D03 dielectric functions; the shorter-reaching of astrosilicate and graphite stops at 6.1992×10^{-5}, past the table's own end
`kext_zubko_BARE_GR_S.dat`	866	0.08998–10^4	the ZDA optics table cut at the Lyman limit
`kext_zubko_BARE_GR_S_euv.dat`†	1201	10^{-3}–10^4	the ZDA optics table itself; this model needs no extension and could not have one

Table 7: Size-integrated transport optics written by `calc_kext.x`. All carry the columns of §7.. † marks the file its builder loads by default (Table 2); the others are there for a host that asks for them through `kext_path`. A non-EUV product is a row subset of its EUV counterpart — same grid, same numbers, over the overlap. `kext_astrodust_MW.dat` is a regression reference, written through its own narrower field widths; it is reproduced byte for byte by `./calc_kext.x astrodust`. Like every other text product it is regenerable and therefore not under version control (`.gitignore`); the tracked form of these optics is the model's `.h5`. Every product's wavelength column carries 8 significant digits rather than the 6 it began with, which the EUV grids need: they resolve each X-ray absorption edge with a pair of points as close as 1.2×10^{-6} in relative wavelength, 6 digits round both to the same string, and a repeated wavelength makes `load_kext_table` reject the file for not ascending. The non-EUV grids never come closer than $\Delta \ln \lambda = 0.01156$ and would be unambiguous at 6, but the field widths are the writer's, not the grid's.